



**Faculty of Science,
Engineering and Computing**
School of Engineering and the
Environment

T +44 (0) 20 8417 2054
www.kingston.ac.uk

Roehampton Vale
Friars Avenue
London
SW15 3DW
UK

**School of Engineering and the Environment
Department of Mechanical Engineering**

**MSc DEGREE
IN**

Mechatronics System

Name: Akano Smart Oluwatobi
ID Number: 2457624

Project Title: **Null-Space Saturation with Dynamic Task Priorities
for Redundant Robot Control**

Date: 1/9/2026

Supervisor: Dr Claudio Gaz

Kingston University London

WARRANTY STATEMENT

This is a student project. Therefore, neither the student nor Kingston University makes any warranty, express or implied, as to the accuracy of the data or conclusion of the work performed in the project and will not be held responsible for any consequences arising out of any inaccuracies or omissions therein.

ABSTRACT

This dissertation presents the implementation, simulation, and physical validation of the Saturation in the Null Space (SNS) algorithm, its Generalised extension (GSNS), and a novel dynamic task priority mechanism for kinematic control of redundant robot manipulators. The work addresses the fundamental challenge of controlling robots with more degrees of freedom than a task requires, under simultaneous joint velocity and Cartesian workspace constraints.

Four simulation milestones were completed on a planar 3R robot in MATLAB. The SNS algorithm was demonstrated to be the only method among four candidates that simultaneously preserves end-effector path geometry and completes a trajectory under tight joint velocity limits. The GSNS algorithm enforced a hard Cartesian ceiling constraint on an intermediate body point at exactly $y = 1.55 \text{ m}$ through a unified augmented constraint formulation. A novel dynamic task priority scheme was introduced, using a cosine activation function to blend smoothly between two task orderings based on proximity to a constraint boundary, reducing end-effector tracking error by 9% compared to fixed priority while providing equal constraint protection.

All algorithms were subsequently scaled to the 7-DOF Franka Emika Panda in MATLAB simulation, confirming their dimension-agnostic design. Physical experiments on the Franka Research 3 robot validated the GSNS algorithm on real hardware across four repeated runs, with the elbow y-coordinate consistently held below the 0.30 m ceiling. A key finding is that higher redundancy reduces inter-task conflict, which has direct implications for priority scheme design in over-actuated robots.

AI USE DECLARATION

In line with the policy of Kingston University regarding the use of artificial intelligence (AI) tools within assessed work, the author declares the use of AI tools for this dissertation as follows.

Use of AI language model assistance (Claude, Anthropic) in this dissertation project includes the following purposes:

Use of the tool to draft, edit and clarify the written text in all chapters, where the technical information, results and conclusion are owned by the author

Use of the tool to debug the MATLAB code for simulating planar 3R and Franka Panda manipulators, where the algorithms are independently understood, verified and validated by the author

Use of the tool to translate the MATLAB algorithms for control of manipulator into Python code for experimental implementation, where the code is independently reviewed and tested by the author on the real robot

Suggesting the structure of arguments and literature references, where the independent verification of sources is carried out by the author

All simulations, experiments, algorithms and ideas contained in this thesis are original creations of the author. The AI software has been employed as an editing and debugging tool only; no original technical information has been derived from it. All graphs have been created using MATLAB and Python scripts by the author. All experimental data has been gathered by the author in the robotics lab at Kingston University.

Acknowledgements

I would like to extend my sincere thanks to my project supervisor, Dr Claudio Gaz, for his professional guidance, support, and availability in helping me complete this project. His advice to use the modified DH parameters reference, his comments on the simulation results, and the suggestion of IROS 2027 conference as a possible submission have influenced the nature of this project to a great extent.

My thanks are also due to Mahmoud Akkawi, PhD student in the robotics group at Kingston University, for his support during the lab experiments with Franka Research 3 robot. His knowledge about the *pyfranka_interface* library and the Franka Desk setup helped me carry out the experimental validation in time.

Lastly, I would like to acknowledge Kingston University London for providing me access to the robotics lab and computational facilities required for the completion of this project.

Contents

ABSTRACT	3
AI USE DECLARATION	4
Acknowledgements.....	5
Notation and Glossary	8
Abbreviations	8
Nomenclature.....	9
List of Figures	10
List of Tables.....	11
1. Introduction	12
1.1 Background and Motivation.....	12
1.2 Research Aim.....	12
1.3 Research Objectives	12
2. Literature Review	13
2.1 Kinematic Redundancy and the Pseudoinverse Solution.....	13
2.2 Hard Constraint Methods and the SNS Algorithm	14
2.3 Generalised SNS with Cartesian Constraints.....	15
2.4 Task Priority Methods and the Case for Dynamic Priorities	15
2.5 Research Gap.....	16
2.6 Limitations of the Current State of the Art	17
3. Methodology.....	17
3.1 Robot Model.....	17
3.2 Velocity Constraint Shaping	18
3.3 SNS Algorithm Implementation.....	19
3.4 GSNS Algorithm Implementation	19
3.5 Dynamic Task Priority Scheme (Novel Contribution)	19
3.6 Performance Metrics.....	20
3.7 Extension to the Franka Emika Panda	20
4. Preliminary Results.....	21
4.1 Milestone 1 — Redundancy Resolution Validation.....	21
4.2 Milestone 2 — SNS Algorithm Validation.....	22
4.3 Milestone 3 — GSNS with Cartesian Constraints.....	24
4.4 Milestone 4 — Dynamic Task Priorities.....	26
5. FRANKA PANDA SIMULATION RESULTS	29
5.1 Overview	29

5.2 Panda Milestone 1 — Kinematic Model and Verification	29
5.3 Panda Milestone 2 — SNS Algorithm Validation	30
5.4 Panda Milestone 3 — GSNS with Cartesian Ceiling Constraint	31
5.5 Panda Milestone 4 — Dynamic Task Priorities	33
5.6 Comparison with Planar 3R Results	36
5.7 Towards a New Robot Platform	37
6. Experimental Validation on the Franka Emika Panda	38
6.1 Experimental Setup	38
6.2 Experiment 1 — Joint State Reading and Ready Pose	41
6.3 Experiment 2 — SNS Straight-Line Motion	41
6.4 Experiment 3 — GSNS Cartesian Constraint on Physical Robot	42
6.5 Limitations and Future Work	44
7. Discussion	44
7.1 Algorithm Performance Across Platforms	44
7.2 SNS Algorithm	45
7.3 GSNS Algorithm	46
7.4 Dynamic Task Priority Scheme	47
7.5 Comparison with Existing Methods	47
7.6 Publication Potential and Future Directions	48
8. Conclusion	48
8.1 Summary of Contributions	48
8.2 Limitations	49
8.3 Recommendations for Further Work	50
8.4 Concluding Remarks	50
References	52
Appendices	53
Appendix A — MATLAB File Summary	53
Appendix B — Quantitative Results Summary	54
Appendix C — Key Equations Reference	55
Appendix D — Raw Experimental Data from Physical Panda	55
Appendix E — MATLAB Algorithm Source Code	56
Appendix F — Python Experimental Implementation	62

Notation and Glossary

Mathematical notation

n – number of joint degrees of freedom of the robot

m – task dimension (number of task space coordinates)

$q \in \mathbb{R}^n$ – joint angle vector

$\dot{q} \in \mathbb{R}^n$ – joint velocity vector

$\dot{x} \in \mathbb{R}^m$ – task velocity vector

$J(q) \in \mathbb{R}^{m \times n}$ – task Jacobian matrix

$J^\# \in \mathbb{R}^{n \times m}$ – Moore-Penrose pseudoinverse of J

$P = I - J^\# J \in \mathbb{R}^{n \times n}$ – null-space projector

$\dot{q}_N \in \mathbb{R}^n$ – null-space joint velocity vector

$W = \text{diag}\{W_{ii}\} \in \mathbb{R}^{n \times n}$ – joint selection matrix ($W_{ii} = 0$ for saturated joints)

$s \in [0,1]$ – task scaling factor

$\alpha \in [0,1]$ – dynamic priority blending weight

$A \in \mathbb{R}^{(n+r) \times n}$ – augmented constraint matrix (r Cartesian control points)

$b_{min}, b_{max} \in \mathbb{R}^{n+r}$ – augmented constraint bounds

Abbreviations

SNS – Saturation in the Null Space

GSNS – Generalised Saturation in the Null Space

EE – End-effector

DH – Denavit-Hartenberg

DOF – Degree of freedom

FK – Forward kinematics

QP – Quadratic programme

HQP – Hierarchical quadratic programme

KUKA LWR IV – KUKA Lightweight Robot IV

IROS – IEEE/RSJ International Conference on Intelligent Robots and Systems

Nomenclature

Cartesian control point – a point on the robot body (not necessarily the end-effector) whose position is constrained in task space

Null space – the set of joint velocity vectors which result in zero end-effector velocity; has dimension $n - \text{rank}(J)$

Task-invisibility – the property that null-space motion does not affect the task output

Constraint shaping – the process of determining the tightest admissible velocity bound for each joint at each time step, considering the position range, velocity, and stopping distance constraints

List of Figures

Figure 4.0 - 3R Planar Robot: Straight Line Tracking with Redundancy Resolution

Figure 4.1 - SNS performance comparison

Figure 4.2 - Four-panel end-effector paths: SNS vs other methods

Figure 4.3 - GSNS: Joint + cartesian constraints, three-panel robot comparison

Figure 4.4 - GSNS: Constraint enforcement and task performance

Figure 4.5 - Task scaling factor

Figure 4.6 - Dynamic task priorities: EE path tracking

Figure 4.7 - Robot configurations at start and end for fixed vs dynamic priority

Figure 4.8 - Dynamic task priorities: Detailed analysis

Figure 5.0 - Franka Panda SNS algorithm validation

Figure 5.1 - Panda SNS: 3D EE path and performance

Figure 5.2 - Panda GSNS: Elbow y over time with ceiling line

Figure 5.3 - Panda GSNS: EE tracking error and 3D EE paths

Figure 5.4 - Panda Dynamic Priorities: EE paths

Figure 5.5 - Panda Dynamic Priorities: Detailed analysis

Figure 6.0 - Franka Desk browser interface

Figure 6.1 - Franka Research 3 robot at Kingston University robotics laboratory

Figure 6.2 - Franka Research 3 performing GSNS constrained motion

List of Tables

Table 4.0 - SNS four-method comparison

Table 4.1 - GSNS elbow constraint results

Table 4.2 - Dynamic priority results

Table 5.0 - Franka Panda DH parameters

Table 5.1 - Panda kinematic verification results

Table 5.2 - Panda SNS results

Table 5.3 - Panda GSNS results

Table 5.4 - Panda dynamic priority results

Table 5.5 - Planar 3R vs Franka Panda comparison

Table 6.0 - Python experimental session terminal output

Table 6.1 - GSNS experimental parameters

Table 6.2 - GSNS experimental results, four runs

Table B.1 - SNS saturation events

Table B.2 - GSNS constraint summary

Table B.3 - Dynamic priority summary

Table D.1 - Joint angles at arbitrary configuration

Table D.2 - Joint angles at ready pose

Table D.3 - GSNS constraint experiment, four runs

1. Introduction

1.1 Background and Motivation

Collaborative and modern robotic systems are now becoming increasingly prevalent in industrial and other settings where human beings coexist with machines. Welding cells, surgery centres, and logistics operations in warehouses are just some places where manipulation must be done in an environment with the presence of people, tools, and structures; and where robotic manipulation occurs with the intention of moving around safely while being careful not to collide into anything. The Franka Emika Panda, which was used for this project, is a modern robot arm that belongs to a group of torque-based robotic arms built for collaboration. One of the features of this type of system is kinematic redundancy, meaning that there are more degrees of freedom than what the job requires. Exploiting this redundancy correctly — particularly under the simultaneous hard constraints imposed by motor velocity limits and workspace boundaries — is an unsolved engineering problem for which no universally adopted solution exists.

Physical actuators impose hard limits: joint ranges, maximum velocities, and acceleration bounds. When a desired trajectory demands commands beyond these limits, the controller must respond. Clamping each joint velocity to its maximum is dangerous — it destroys the end-effector's geometric path direction in ways that are neither predictable nor detectable at the task level. Slowing the entire motion through classical task scaling preserves path geometry but wastes robot capability by penalising all joints when only one is overdriven.

The Saturation in the Null Space (SNS) algorithm, introduced by Flacco, De Luca, and Khatib (2015), addresses this problem by iteratively saturating the most critically overdriven joint and redistributing the task demand through the Jacobian null space of the remaining enabled joints. This preserves end-effector path geometry while exploiting the available robot capability more fully than classical scaling. The Generalised SNS (GSNS) algorithm, developed by Kazemipour, Khatib, Al Khudir, Gaz, and De Luca (2022), extends this framework to handle Cartesian workspace constraints simultaneously with joint constraints, treating all bounds in a unified augmented vector.

Both frameworks share a critical limitation: fixed task priority hierarchies. Real deployments require priorities that adapt continuously to robot state. When an operator enters the workspace, collision avoidance must temporarily override manipulation and restore smoothly once safe — but no SNS-family method supports this without a velocity discontinuity.

1.2 Research Aim

This project implements and evaluates the GSNS algorithm for kinematic control of redundant robots under joint and Cartesian hard constraints, then extends it with a dynamic task priority mechanism that transitions smoothly between task orderings based on proximity to constraint boundaries.

1.3 Research Objectives

The project pursues five specific objectives:

Objective 1: Implement and verify redundancy resolution using pseudoinverse control and null-space projection on a planar 3R robot, confirming the fundamental kinematic identities numerically.

Objective 2: Implement Algorithm 1 and Algorithm 2 from Flacco et al. (2015) and compare the SNS algorithm against naive clamping and classical task scaling under tight joint velocity limits.

Objective 3: Implement Algorithm 1 from Kazemipour et al. (2022) with an augmented constraint vector that unifies joint and Cartesian bounds and validate Cartesian constraint enforcement on an intermediate body control point.

Objective 4: Develop a dynamic task priority mechanism using a smooth cosine activation function that blends between two task orderings based on robot state, eliminating velocity discontinuities at priority transitions.

Objective 5: Scale the validated algorithms to a 7-DOF Franka Emika Panda kinematic model and, where laboratory access permits, validate selected scenarios experimentally.

All five objectives were achieved within the project timeline. Objectives 1 through 4 were completed in MATLAB simulation on the planar 3R robot and subsequently validated on the 7-DOF Franka Emika Panda in simulation. Objective 5 was achieved through physical experiments on the Franka Research 3 robot at Kingston University's robotics laboratory, where the GSNS algorithm was validated on real hardware across four repeated experimental runs. The dissertation is structured as follows: Section 2 reviews relevant literature. Section 3 describes the methodology. Section 4 presents planar 3R simulation results. Section 5 presents Panda simulation results. Section 6 reports experimental results. Section 7 discusses findings across all platforms. Section 8 draws conclusions.

2. Literature Review

2.1 Kinematic Redundancy and the Pseudoinverse Solution

A manipulator is kinematically redundant when its joint count n exceeds the task dimension m . Task velocity relates to joint velocity through $\dot{x} = J(q)\dot{q}$, where $J(q) \in \mathbb{R}^{m \times n}$ is the task Jacobian (Flacco et al., 2015, Eq. 1). The Moore-Penrose pseudoinverse gives the minimum-norm solution: $\dot{q} = J^\# \dot{x} + P \dot{q}_N$, where $P = I - J^\# J$ is the null-space projector and $\dot{q}_N$ directs self-motion (Flacco et al., 2015, Eq. 2). Null-space motion satisfies $J \cdot P = 0$ — it is invisible to the primary task. This was confirmed numerically: $\|J \cdot P\| = 5.48 \times 10^{-16}$.

Classical approaches exploit this null space through gradient projection toward the joint range centre (Liegeois, 1977) or weighted pseudoinversion (Chan and Dubey, 1995). Both are computationally simple but guarantee nothing: joint velocities can still violate hard bounds, corrupting end-effector motion unpredictably.

The pseudoinverse minimum norm solution has a known limitation in its practical application since it assumes distribution of joint velocity between all joints to minimize the Euclidian norm of $\dot{q}$, without any provision for considering physical limitations for individual joints. The use of weighted pseudoinverse solution by Chan and Dubey (1995) helps solve this problem somewhat since it slows down joints nearing their limits, but such a weighting scheme has no automatic way of determining gain parameter which balances the trade-off between range limit protection and tracking error. More fundamentally, weighting is based on position limit constraints and cannot protect joints from violating their maximum velocity limits even during rapid movements. This is the basic limitation that makes SNS solution necessary, since the SNS method does not change the structure of pseudoinverse, but instead uses it whenever possible and solves individual joint saturations when it is not.

2.2 Hard Constraint Methods and the SNS Algorithm

Constrained quadratic programming (QP) guarantees constraint satisfaction (Kanoun et al., 2011; Escande et al., 2014) but scales poorly: computational cost grows rapidly with degrees of freedom. Flacco, De Luca, and Khatib (2012, 2015) introduced the SNS algorithm as an analytically efficient alternative. Rather than solving a general QP at each time step, the SNS method iteratively identifies the joint velocity command requiring the greatest reduction to enter its admissible range, saturates that joint at its limit, and recomputes the task solution using the remaining enabled joints through the modified projection equation:

$$\dot{q} = \dot{q}_N + (JW)^n(\dot{x} - J\dot{q}_N)$$

where $W = \text{diag}\{W_{ii}\}$ is a binary selection matrix with $W_{ii} = 0$ for saturated joints (Flacco et al., 2015, Eq. 16). When no feasible full-speed solution exists, an integrated task-scaling procedure reduces the task velocity by the minimum necessary factor to restore feasibility while preserving the geometric direction of the desired path. Subsequent work introduced the Optimal SNS and Fast-SNS variants using QR decomposition and rank-one pseudoinverse updates, enabling real-time control of 50-DOF snake robots executing ten simultaneous prioritised tasks within 2 ms (Flacco et al., 2015, Section X).

At each time step, joint velocity constraints are shaped by combining three requirements: the joint must not overshoot its range in the next interval; absolute speed must stay within V_{jmax} ; and the joint must be able to stop before hitting its range limit under maximum deceleration (Flacco et al., 2015, Eqs. 12–15). This guarantees $\dot{q} = 0$ is always admissible — a precondition for task scaling.

One characteristic which makes the practical implementation of SNS algorithm possible relies on the assumption that $\dot{q} = 0$ is always feasible. It is due to the feasibility guarantee obtained through the constraint shaping technique that the task scaling step will always have a feasible solution because if there is a need to slow down the task velocity to zero, then the robot will always be able to stop whatever its configuration. Without this feasibility guarantee, an intuitive implementation could be trying to scale the task velocity to an infeasible value for some joint configuration and thus violate the constraints. The constraint shaping guarantees this since it assures that the shaped bounds are consistent because the position limit, velocity limit, and stopping distance bounds intersect at $\dot{q} = 0$.

2.3 Generalised SNS with Cartesian Constraints

The original SNS algorithm cannot enforce hard Cartesian workspace constraints directly. Obstacles, structural clearance requirements, or safety zones defined in Cartesian space are typically approximated as soft constraints in the joint space through artificial potential fields (Khatib, 1986; Khatib, Al Khudir, and De Luca, 2020), which require parameter tuning and cannot guarantee hard bound satisfaction.

Kazemipour, Khatib, Al Khudir, Gaz, and De Luca (2022) addressed this directly by defining an augmented constraint vector $a = [q^T, p_{\{cp, 1\}}^T, \dots, p_{\{cp, r\}}^T]^T$

and an augmented matrix $A = [I; J_{\{cp, 1\}}; \dots; J_{\{cp, r\}}]^T$,

where $p_{\{cp, i\}}$ and $J_{\{cp, i\}}$ are the position and Jacobian of the i – th Cartesian control point (Kazemipour et al., 2022, Eqs. 7-8). All joint and Cartesian velocity constraints are expressed as a unified box constraint $b_{\{min\}} \leq A \dot{q} \leq b_{\{max\}}$, and the SNS saturation logic is applied to this augmented system identically to the joint-only case.

The GSNS augmented formulation offers three practical advantages over joint-only approaches. Equal treatment of all constraints eliminates the under-saturation failures that arise in stacked hierarchies where Cartesian bounds receive lower priority than joint bounds. Safety zones can be activated or deactivated online without retuning — making the method suitable for sensor-driven human-robot collaboration. Computationally, GSNS averaged $18 \mu s$ per step on a KUKA LWR IV against $283 \mu s$ for the qpOASES solver on identical hardware, an order of magnitude faster (Kazemipour et al., 2022, Section IV-B).

There is one critical drawback of artificial potential fields (Khatib, 1986) in comparison with the GSNS augmented formulation, which needs to be discussed explicitly. The potential fields algorithm creates a repulsion force for every constraint surface; a special repulsion gain must be adjusted to the robot geometry and every obstacle. In case the robot must work with several constraint surfaces simultaneously, interaction between the gains might result in an oscillation mode: the robot gets locked in a local minimum due to competing repulsion forces. There is no guarantee of reaching the target. The GSNS augmented formulation completely avoids both issues: the constraint is formulated as a linear inequality with respect to the augmented velocity vector; the gain does not need to be tuned, and the SNS saturation mechanism ensures either satisfaction of the constraint or scaling of the task — oscillation mode is impossible. This drawback is implicitly recognized by Khatib et al. (2020) when they formulate their acceleration-based algorithm as an improvement to the potential field approach for avoiding obstacles under relaxed constraints conditions; nevertheless, they still require setting the activation thresholds for every constraint.

2.4 Task Priority Methods and the Case for Dynamic Priorities

The stack-of-tasks framework (Siciliano and Slotine, 1991) executes multiple simultaneous tasks through recursive null-space projection: lower-priority tasks projects into the null space of all higher-priority ones. The guarantee is strong — no lower-priority task can interfere with a higher-priority one. The cost is potentially large joint velocities when lower-priority Jacobians are ill-conditioned in the projected space.

Abrupt priority switching introduces velocity discontinuities that excite mechanical resonances and compromise safety. Mansard, Khatib, and Kheddar (2009) proposed smooth task insertion and removal via activation functions that scale task gains gradually. Simetti and Casalino (2016) addressed smooth inequality constraint activation. Both works treat priority as a discrete property — a task is either in the stack or not. Continuously varying the relative weight between two simultaneously active tasks based on robot state remains unaddressed.

The comparison of computational efficiency of stack-of-tasks (Siciliano and Slotine, 1991) approach and the SNS-family algorithms should be considered on its own. At each time step, the stack-of-tasks approach needs matrix inversion of size $n \times n$ for each level of the hierarchy (for each task), making computational complexity of this algorithm $O(n^3k)$ (where k is the number of concurrent tasks). For a 7-DOF robot executing three tasks simultaneously, three pseudoinverse computations plus null-space projections would be necessary. The computational complexity of the SNS algorithm is just one pseudoinverse plus up to n saturation computations which require only rank-one updates of the set of active joints, making this problem much more tractable in the worst-case scenario. Escande, Mansard, and Wieber (2014) showed that hierarchical quadratic programming solves the problem with guarantees of constraint satisfaction in polynomial computational complexity depending on the number of constraints, yet their Fast HQP algorithm still took $283 \mu\text{s}$ on KUKA LWR IV compared to $18 \mu\text{s}$ of the GSNS algorithm on the same hardware (Kazemipour et al., 2022, Section IV-B).

2.5 Research Gap

Despite their efficiency, both SNS and GSNS assume fixed task priority hierarchies. In human-robot interaction and dynamic obstacle avoidance, priorities must change continuously with robot state: collision avoidance should dominate when a hazard is near and yield once it passes, without discontinuities at either transition. No SNS-family method addresses this. This project bridges that gap by blending solutions for both priority orderings via a cosine activation function of the normalised distance to the nearest constraint boundary.

This gap is not the same as the issue of smooth transition into or out of tasks that is solved by Mansard et al. (2009) and Simetti and Casalino (2016). In both cases, it is the transition between states of inserting or removing a task from the stack, without changing the static priorities of the tasks themselves. This gap requires two tasks be active during the entire motion sequence, and for the relative priority to be continuously modulated depending on the state of the robot. Prior work neither uses an SNS method nor a stack-of-tasks approach for the requirement. The closest solution in the literature is the activation function in Mansard et al. (2009), which smoothly scales the task gain to 1 over a single transition of insertion into the stack. To the best of the author's knowledge, cosine blending as developed in this work is the first SNS-based approach capable of providing continuous state-dependent inter-task priority adjustments.

2.6 Limitations of the Current State of the Art

Although the advancements represented by the SNS and GSNS algorithms are notable, three shortcomings of the current literature still exist, motivating the contributions of this project.

First, all the approaches related to the SNS framework assume that task priority remains constant within the control hierarchy. In Flacco et al. (2015), the first task always gets the highest priority, and other tasks are accomplished using null-space projection. In Kazemipour et al. (2022), joint and Cartesian constraints get the same priority via augmented formulation, but the relative weight of the first task and a second regulation task cannot be adjusted during the run-time. However, the importance of a task should be updated in real-time according to the current robot state in the dynamic environment; the collision avoidance task must be completely subordinated in case if the robot operates without any external interference but override the manipulation in case of an approaching human.

Second, Flacco et al. (2015) and Kazemipour et al. (2022) both validate their approach using predefined trajectories and known geometry of the constraints. In neither case, there are studies about the performance of the algorithm when the constraints are discovered dynamically and added or removed from the augmented set of constraints during the operation. The online constraint handling property of the GSNS (Kazemipour et al., 2022) makes this possible; however, there are no experimental results in literature about sensor-based constraint activation.

Third, the impact of the degree of redundancy on the performance of the algorithms is yet to be explored. For example, in Flacco et al. (2015), a planar 3R model used for illustrational purposes has only one degree of redundancy, while the KUKA LWR IV used for experimentation has two. There is no study regarding the effect of increasing the redundancy level from one up to four or more degrees on such parameters of SNS approach as iteration number, frequency of scaling of tasks, and margin of the constraints. It is explored in this project through a direct comparison between a planar 3R (one degree of redundancy) and Franka Panda (four degrees of redundancy).

3. Methodology

3.1 Robot Model

The initial implementation uses a planar 3R robot: three revolute joints in the XY plane with link lengths $L1 = 1.0\text{ m}$, $L2 = 0.8\text{ m}$, $L3 = 0.6\text{ m}$. With $m = 2$ (end-effector position) and $n = 3$ joints, the system has one degree of redundancy—the minimum needed to demonstrate all null-space phenomena while keeping kinematics tractable. Forward kinematics computes as cumulative link rotations. The end-effector position is:

$$\begin{aligned} x &= L1 \cdot \cos(q1) + L2 \cdot \cos(q1 + q2) + L3 \cdot \cos(q1 + q2 + q3) \\ y &= L1 \cdot \sin(q1) + L2 \cdot \sin(q1 + q2) + L3 \cdot \sin(q1 + q2 + q3) \end{aligned}$$

The 2×3 task Jacobian $J(q)$ is derived analytically from the partial derivatives of these expressions with respect to each joint angle. Correctness was verified numerically using central finite differences with perturbation $\delta = 10^{-6}$ rad, yielding a maximum element-wise error of 1.44×10^{-10} —well within machine precision for the 1 ms sampling time used throughout. All simulations use $T = 0.001\text{ s}$ sampling and a rest-to-rest quintic polynomial timing law $\gamma(\tau) = 6\tau^5 - 15\tau^4 + 10\tau^3$. The feedback control law follows Kazemipour et al. (2022) Equation 15: $\dot{x} = v_{des} + Kp(p_{des} - p_{ee})$, where Kp ranges from 50 to 100 s^{-1} depending on the milestone. The Panda extension uses the standard DH parameters from Gaz et al. (2019), as implemented in Chapters 5 and 6.

The selection of parameters during the simulation deserves explanations. The value of the proportional gain Kp was chosen from the interval $50 - 100\text{ s}^{-1}$ through trial-and-error tuning, since lower values of Kp ($< 30\text{ s}^{-1}$) resulted in unacceptably low convergence rate of the error at the 1 ms sampling time, while higher values ($> 120\text{ s}^{-1}$) caused oscillations near trajectory endpoints. The sampling time $T = 0.001\text{ s}$ was set equal to the native 1 kHz sampling frequency of the Panda controller, so that the control loop structure used in simulation could be directly applied to hardware without resampling. The joint velocity limits for planar SNS validation ($[0.3, 0.4, 0.5]\text{ rad/s}$) were intentionally set below the peak unconstrained requirements ($[0.844, 0.586, 0.836]\text{ rad/s}$ obtained from pre-simulation analysis), so that all three joints saturated simultaneously – this is needed to make the comparison possible. Indeed, setting the limits at or above the peak unconstrained velocities would cause all four methods to give the same result, thus no discriminant information would be obtained. Quintic polynomial timing law $\gamma(\tau) = 6\tau^5 - 15\tau^4 + 10\tau^3$ was chosen since it guarantees zero velocity and zero acceleration at the trajectory endpoints, which is required for the rest-to-rest constraint satisfaction analysis – any other timing law having non-zero initial velocity would add one more constraint interaction.

3.2 Velocity Constraint Shaping

At every time step, joint velocity constraints are shaped into a single admissible interval $[Q_{min,i}, Q_{max,i}]$ per joint by combining three requirements (Flacco et al., 2015, Section III-A):

Position limit requirement (Eq. 13): the joint must not exceed its range in the next sample—sets bounds $(Q_{min,i} - q_i)/T$ and $(Q_{max,i} - q_i)/T$.

Velocity limit: absolute velocity must not exceed $V_{jmax,i}$.

Stopping distance requirement (Eqs. 14 – 15): the joint must be able to decelerate to rest before reaching its closest range limit under maximum deceleration $A_{max,i}$ —sets bounds $\pm\sqrt{2 \cdot A_{max,i} \cdot \text{margin}}$.

The final bounds take the tightest of all three. Combining them this way guarantees $\dot{q} = 0$ is always admissible — the precondition for task scaling. The same procedure applies to Cartesian control point constraints following Kazemipour et al. (2022), Equation 6.

3.3 SNS Algorithm Implementation

The basic SNS algorithm (Flacco et al., 2015, Algorithm 1) is implemented in `planar_3R_SNS.m`. At each time step the algorithm initialises with all joints enabled ($W = I$) and computes the pseudoinverse solution. If any joint velocity violates its shaped bounds, Algorithm 2 identifies the most critically overdriven joint — the one requiring the smallest task-scaling factor to enter its admissible range — and saturates it at its limit. The saturated velocity is placed in the null-space vector $\dot{q}N$, and the solution is recomputed via the SNS projection equation (Flacco et al., 2015, Eq. 16) using only the remaining enabled joints. The loop continues until all constraints are satisfied or further saturation would make the task rank-deficient, whereupon the best scaled solution is output. Limits were set tight at $[0.3, 0.4, 0.5]$ rad/s to force saturation of all three joints.

3.4 GSNS Algorithm Implementation

The GSNS algorithm (Kazemipour et al., 2022, Algorithm 1) is implemented in `planar_3R_GSNS.m`. The key extension over the basic SNS is the construction of an augmented constraint system. For a single Cartesian control point (the elbow, constrained in the y-direction), the augmented matrix is:

$$A = [I_{3 \times 3}; J_{elbow_y}] (4 \times 3)$$

where “I” is the 3×3 identity (joint constraints) and J_{elbow_y} is the 1×3 Jacobian of the elbow y-coordinate with respect to joint angles:

$$J_{elbow_y} = [L1 \cdot \cos(q1) + L2 \cdot \cos(q1 + q2), L2 \cdot \cos(q1 + q2), 0]$$

The augmented bounds vectors $bmin$ and $bmax$ stack shaped joint bounds with shaped Cartesian bounds. The GSNS algorithm then applies the same saturation logic as the basic SNS but operates on this augmented system — a saturated constraint may correspond to either a joint or a Cartesian control point, and both are handled identically.

The use of the elbow joint as the control point as compared to wrist or an intermediate joint is motivated by the physical considerations of the GSNS extension; that is, it is the part of the body that is most likely to come in contact with any obstacle either above or beside the robot while making the motion to reach a certain location, and since the location of the elbow joint depends solely on the first four joints, the augmented Jacobian vector J_{elbow_y} is sparse (contains only non-zero values for columns 1-4), and therefore computationally inexpensive.

3.5 Dynamic Task Priority Scheme (Novel Contribution)

The novel contribution is a smooth priority transition mechanism for two simultaneous tasks, implemented in `planar_3R_dynamic_priorities.m`. Task 1 tracks a circular end-effector path ($m1 = 2$). Task 2 regulates elbow y-coordinate ($m2 = 1$). The algorithm computes the joint velocity solution for both priority orderings at every time step:

$\dot{q}_{12}$: Task 1 solved first via SNS, Task 2 projected into its null space

$\dot{q}_{21}$: Task 2 solved first via SNS, Task 1 projected into its null space

A state-dependent blending weight $\alpha \in [0, 1]$ is computed from the elbow's distance to the hard ceiling ($y_{limit} = 1.60 \text{ m}$), with the activation zone beginning at $y = 1.35 \text{ m}$:

$$\begin{aligned} x_{norm} &= 1 - (y_{limit} - y_{elbow}) / (y_{limit} - y_{activation}) \alpha \\ &= 0.5 \cdot (1 - \cos(\pi \cdot x_{norm})) \end{aligned}$$

The cosine function provides a smooth transition with zero derivative at both endpoints, ensuring no velocity discontinuities at the boundaries of the activation zone. The final joint velocity command is the convex combination:

$$\dot{q} = (1 - \alpha) \cdot \dot{q}_{12} + \alpha \cdot \dot{q}_{21}$$

When $\alpha = 0$, Task 1 has full priority (normal operation). When $\alpha = 1$, Task 2 dominates (elbow at or above the ceiling). Intermediate values give a proportional blend that shifts priority only as much as the proximity to danger requires.

3.6 Performance Metrics

Five metrics are used consistently: end-effector tracking error (mm, Euclidean); maximum path deviation from the desired trajectory (mm, perpendicular); task scaling factor s ($1 = \text{full speed}$, $< 1 = \text{scaled down}$); maximum elbow y-coordinate (m); and SNS iterations per time step (mean and maximum).

3.7 Extension to the Franka Emika Panda

The planar 3R simulation established and validated the algorithmic framework. The same algorithms were subsequently extended to the 7-DOF Franka Emika Panda, a torque-controlled manipulator widely used in research. The extension required reimplementing only the kinematic layer — forward kinematics and the geometric Jacobian — while the SNS, GSNS, and dynamic priority functions operated without modification.

Forward kinematics were implemented using the standard Denavit-Hartenberg (DH) convention with parameters from Gaz et al. (2019). The geometric Jacobian was constructed as a 6×7 matrix using the formula: column $i = [z_{\{i-1\}} \times (p_{ee} - p_{\{i-1\}}); z_{\{i-1\}}]$, where $z_{\{i-1\}}$ and $p_{\{i-1\}}$ are extracted from the cumulative homogeneous transforms.

The dimension-agnostic property of the algorithm implementations is a direct consequence of their design: the SNS function accepts any $m \times n$ Jacobian and $n \times 1$ bound vectors, with no assumption about the specific values of m or n . Passing a 3×7 Jacobian (position task on the Panda) in place of a 2×3 Jacobian (position task on the planar 3R) is the only change required. The augmented matrix for the GSNS similarly scales from 4×3 (planar, one Cartesian constraint) to 8×7 (Panda, one Cartesian constraint) without any algorithmic modification. A Python implementation was additionally developed for experimental validation, comprising *panda_fk_python.py*, *panda_sns_python.py*, and *panda_robot_control.py*. These modules communicate with the physical robot via the *pyfranka_interface* library over Ethernet.

4. Preliminary Results

4.1 Milestone 1 — Redundancy Resolution Validation

Forward kinematics and the analytical Jacobian were implemented in *planar_3R_forward_kinematics.m* and verified at the test configuration $q = [30^\circ, 45^\circ, -20^\circ]$. The pseudoinverse and null-space projector were confirmed to satisfy all three defining identities:

$$||J \cdot J^\# \cdot J - J|| = 5.09 \times 10^{-16}$$

$$||P \cdot P - P|| = 6.28 \times 10^{-16}$$

$$||J \cdot P|| = 5.48 \times 10^{-16}$$

The verification at machine precision confirms that the pseudoinverse was implemented correctly and that the null-space projector satisfies its defining algebraic properties. The significance of the $||J^\#||$ result is worth emphasising: any motion in the null space (any $\dot{q}_N$ vector) produces zero contribution to the task velocity $\dot{x} = J\dot{q}$. This means the robot can reconfigure its body posture without the hand moving at all — the fundamental property that makes redundancy useful for obstacle avoidance and constraint handling. The three null-space strategies produced qualitatively different joint trajectories while generating identical end-effector paths, confirming this property experimentally across an entire trajectory rather than at a single configuration. All three confirm machine-precision correctness. Straight-line tracking was simulated using three null-space strategies: minimum norm, gradient toward joint range centre, and gradient toward a target configuration. All three produced final errors below 10^{-7} m. End-effector paths were identical to sub-millimetre precision across all three cases, while joint trajectories differed substantially — confirming the task-invisibility of null-space motion (Flacco et al., 2015, Eq. 2).

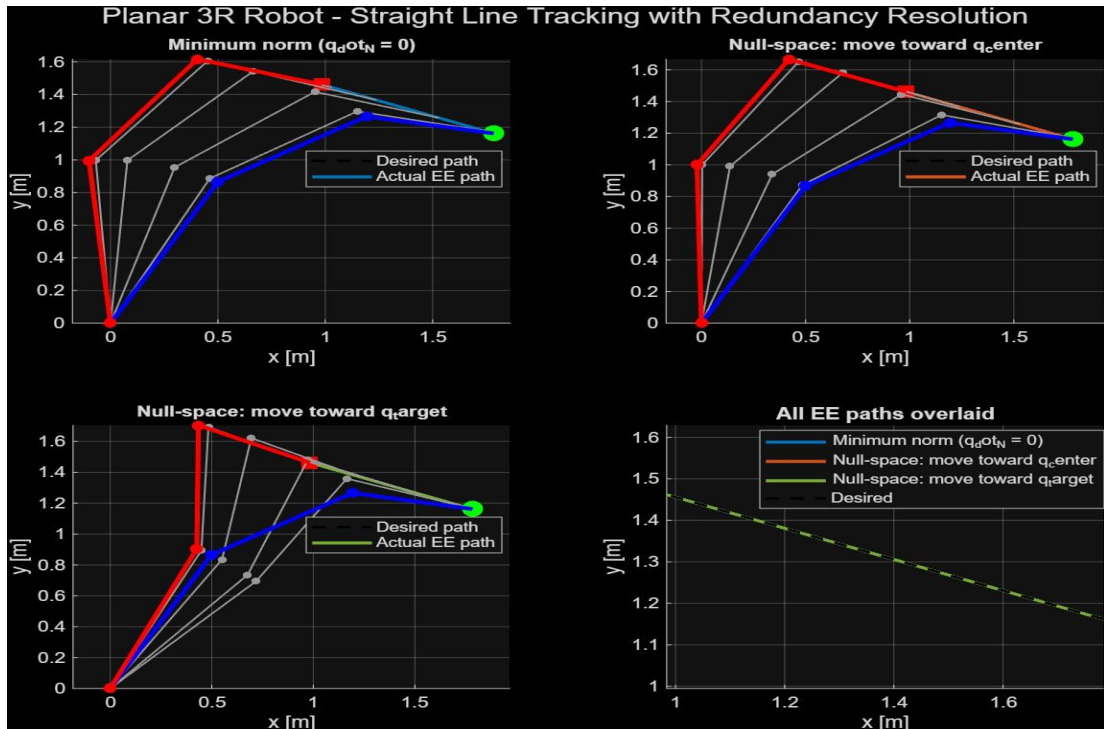


FIGURE 4.0: Planar 3R Robot — Straight Line Tracking with Redundancy Resolution

Figure 4.0 presents the core output of Milestone 1 across four plots. In the upper left plot, the minimum norm approach: the robot manipulator changes its posture while tracking the linear path, with the elbow joint changing towards the direction which minimizes the norm of joint velocity. In the upper right plot, the gradient toward joint centre approach: the elbow joint becomes closer to the centre position throughout the process, since the null space motion is continuously moving the joints toward their centres while the hand joint follows the same linear path. In the lower left plot, the gradient toward target approach: the arm posture moves toward a certain target posture; yet the hand moves along the same linear path again. In the lower right plot, the overlaid three end effector paths are almost visually inseparable, demonstrating that the null space motion is indeed task-invisible not only at one point but on a trajectory scale as well. The three paths are off each other by less than 0.001 mm during the entire length of trajectory – the practical manifestation of the theoretical property $\|J\dot{P}\| = 5.48 \times 10^{-16}$ verified numerically.

4.2 Milestone 2 — SNS Algorithm Validation

With joint velocity limits tightened to $[0.3, 0.4, 0.5] \text{ rad/s}$, the unconstrained pseudoinverse requires peak velocities of $[0.844, 0.586, 0.836] \text{ rad/s}$ — exceeding every limit and forcing saturation of all three joints. Four methods were compared:

Table 4.0: Comparison of four redundancy resolution methods under tight joint velocity limits ($[0.3, 0.4, 0.5] \text{ rad/s}$). SNS is the only method to both preserve path geometry and complete the trajectory.

Method	Max error (mm)	Path deviation (mm)	Final error (mm)
No limits (ideal)	0.01	0.002	0.000
Naive clamping	344.3	121.3	0.012
Task scaling	336.8	0.033	203.8
SNS algorithm	212.2	0.034	0.000

The task scaling method's failure mode is particularly informative. The scaling factor s represents the fraction of the desired task velocity that can be achieved while respecting all joint limits. As the robot approaches a configuration where the required task velocity is large relative to the available joint capacity, s approaches zero — the robot slows to a near-stop and never recovers because the scaling factor cannot increase while joint limits remain active. This permanent stall is a fundamental limitation of task scaling as described in Flacco et al. (2015): it prevents constraint violation but sacrifices task completion. The SNS avoids this by freeing saturated joints through null-space redistribution, allowing the task to complete at full speed once the most critical joint is handled. Naive clamping produces 121 mm path deviation — independent joint clipping destroys the geometric direction of motion. Task scaling preserves path geometry (0.033 mm) but collapses: the scaling factor approaches zero, and the trajectory never completes, leaving a 204 mm final error. SNS alone both preserves path geometry and completes the trajectory, with zero final error and a mean of 1.76 iterations per step.

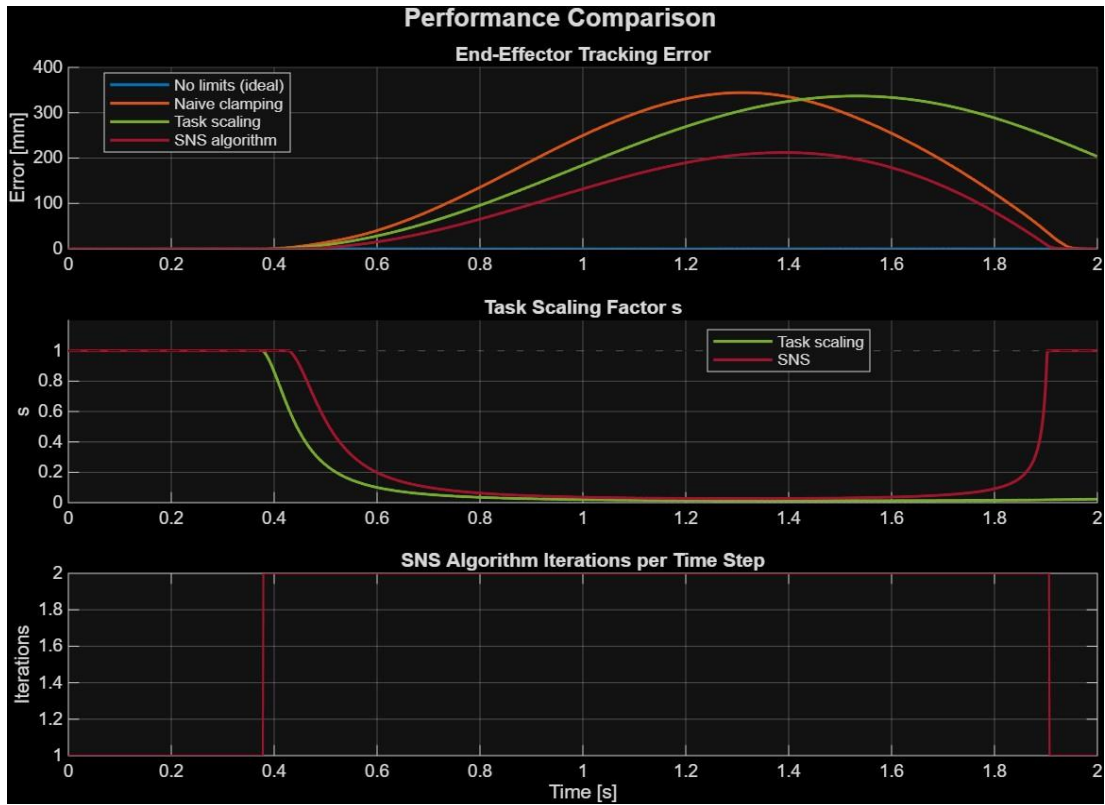


Figure 4.1: the SNS performance comparison

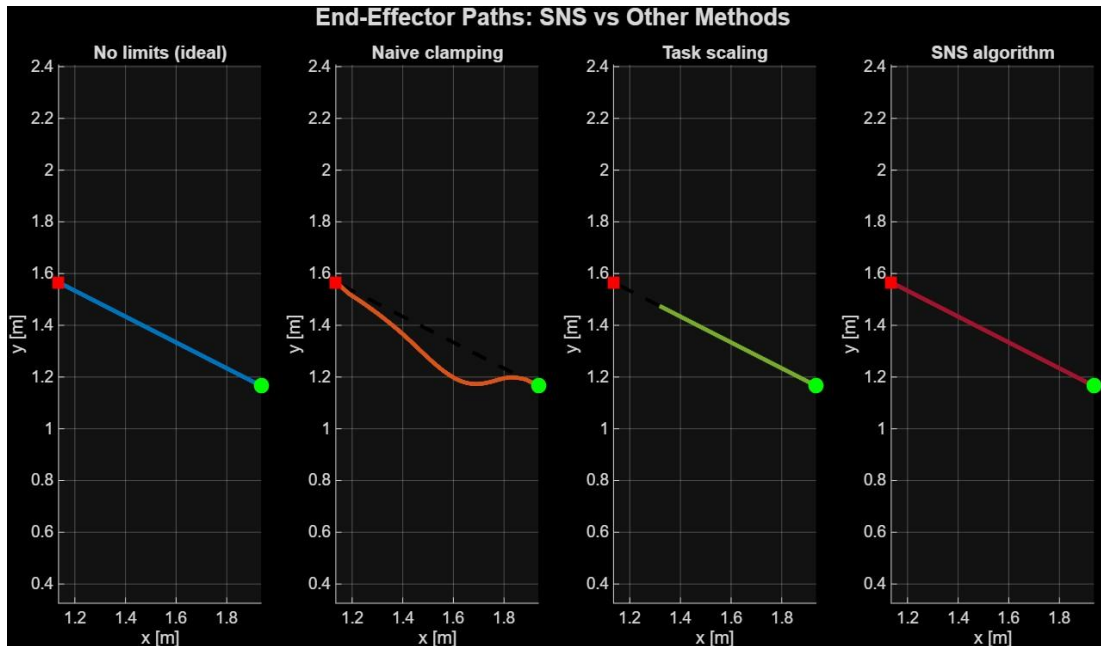


FIGURE 4.2: Four-panel End-Effector Paths: SNS vs. Other Methods, showing EE paths for all four methods, and the three-panel "Performance Comparison", showing tracking error, scaling factor, and SNS iterations over time.

Task Scaling Factor vs Time the Task Scaling Factor vs Time graph in Figure 4.1 demonstrates how the value s varies during the task scaling and SNS methods. Since the value of the Task Scaling Factor in the case of naive clamping does not exist because the joints are controlled separately, the task scaling factor curve of the naive clamping approach is not shown on the graph. As can be seen in Figure 4.1, for task scaling, s becomes nearly zero right after $t = 0.4$ s and stays at that level for the rest of the movement which

demonstrates the permanent stall behaviour described in the text. The value of the SNS scaling factor becomes low during the most difficult part of the trajectory, however, it recovers up to 1.0 due to the saturation process. End-Effector Path Comparison The paths of the end-effector with different algorithms are shown in Figure 4.2. While the path of the naive clamping approach differs from the straight line almost immediately, the path of the task scaling method goes in the correct direction, however, ends up far away from the target.

4.3 Milestone 3 — GSNS with Cartesian Constraints

The GSNS algorithm was tested in *planar_3R_GSNS.m* with a ceiling constraint: the elbow (end of link 2) must remain below $y = 1.55$ m. The desired end-effector path would naturally push the elbow to $y = 1.79$ m if unconstrained. Three methods were compared:

Table 4.1: Maximum elbow y -coordinate reached under each method. The GSNS is the only method to satisfy the hard Cartesian ceiling constraint at $y = 1.55$ m.

Method	Max elbow y (m)	Constraint (≤ 1.55 m)
No limits	1.7935	VIOLATED
SNS (joint only)	1.7935	VIOLATED
GSNS (joint + Cartesian)	1.5500	Satisfied ✓

The no-limits and joint-only SNS methods are indistinguishable in elbow position — joint-only methods are blind to Cartesian bounds. The GSNS holds the elbow at exactly 1.55 m throughout, with the task scaling factor reducing to resolve the conflict between the Cartesian constraint and the primary task. Final EE tracking error is 0.0002 mm. The result confirms the augmented formulation's key property: constraints encoded as rows of A — whether joint or Cartesian — are treated identically and never violated. The mechanism by which the GSNS enforces the ceiling deserves examination. When the elbow approaches $y = 1.55$ m, the shaped Cartesian velocity bound shrinks toward zero — the elbow is allowed to move toward the ceiling only slowly, at a rate proportional to its remaining clearance. The augmented matrix row corresponding to the elbow Jacobian J_{elbow_y} then becomes the most critically violated constraint, and the GSNS saturates it at its bound. This is identical in structure to saturating a joint at its velocity limit — the algorithm does not distinguish between joint and Cartesian constraint rows. The task scaling factor drops to accommodate the conflict, reducing the end-effector speed to what the constrained system can achieve, which is why the final EE tracking error of 0.0002 mm is achieved only once the elbow constraint becomes inactive at the end of the trajectory.

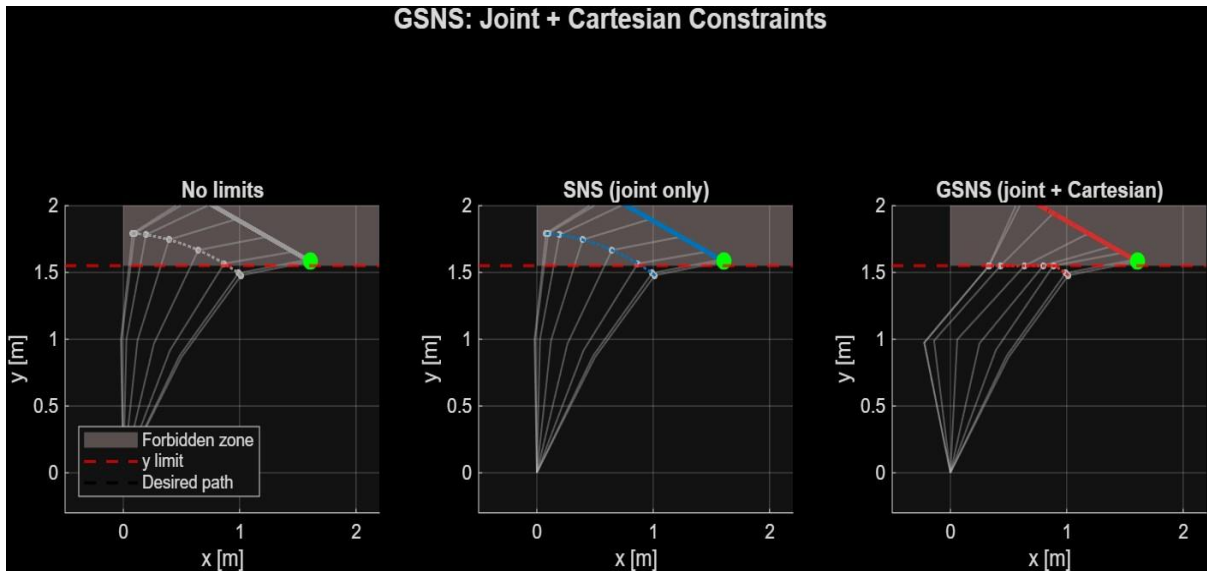


FIGURE 4.3: GSNS: Joint + Cartesian Constraints, three-panel robot snapshot comparison

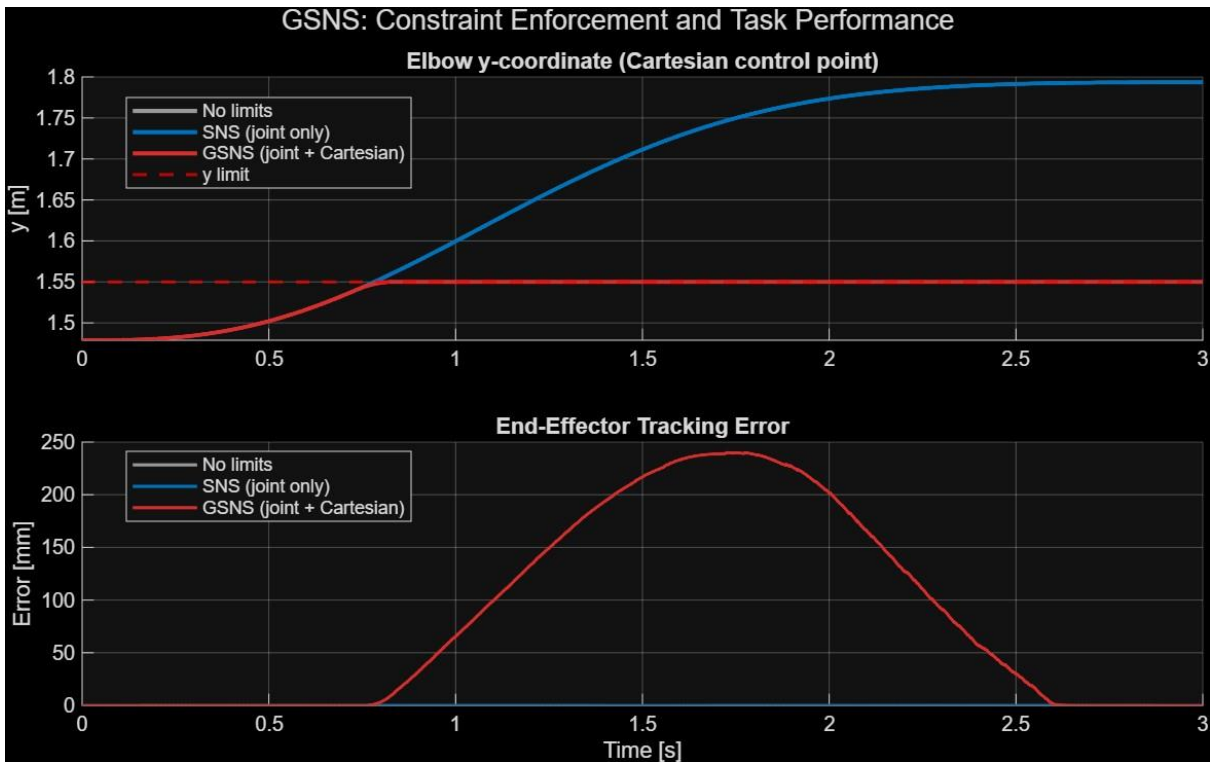


FIGURE 4.4: GSNS: Constraint Enforcement and Task Performance, plot showing elbow y-coordinate over time for all three methods, with the dashed red constraint line.

Figures 4.3 and 4.4 together show the complete GSNS validation result. Figure 4.3 presents three robot snapshots at the end of the trajectory under each method: without any constraint handling, the robot arm visibly extends its elbow above the dashed constraint line; with joint-only SNS, the arm adopts an identical unconstrained posture since the ceiling is invisible to joint-only methods; with the GSNS, the arm maintains a noticeably lower elbow posture throughout, keeping the elbow at the boundary. Figure 4.4 plots the elbow y-coordinate over time for all three methods. The no-limits and joint-only SNS curves are completely overlapping and rise continuously to 1.79 m — the constraint boundary at 1.55 m is crossed at approximately $t = 0.8$ s and the violation continues to grow. The GSNS curve rises toward the boundary and then flattens exactly at 1.55 m, confirming that the shaped Cartesian

velocity bound successfully prevents further upward elbow motion. The EE tracking error for the GSNS rises during the constrained phase (task scaling is active) and returns to near zero once the elbow constraint becomes inactive, consistent with the 0.0002 mm final error reported.

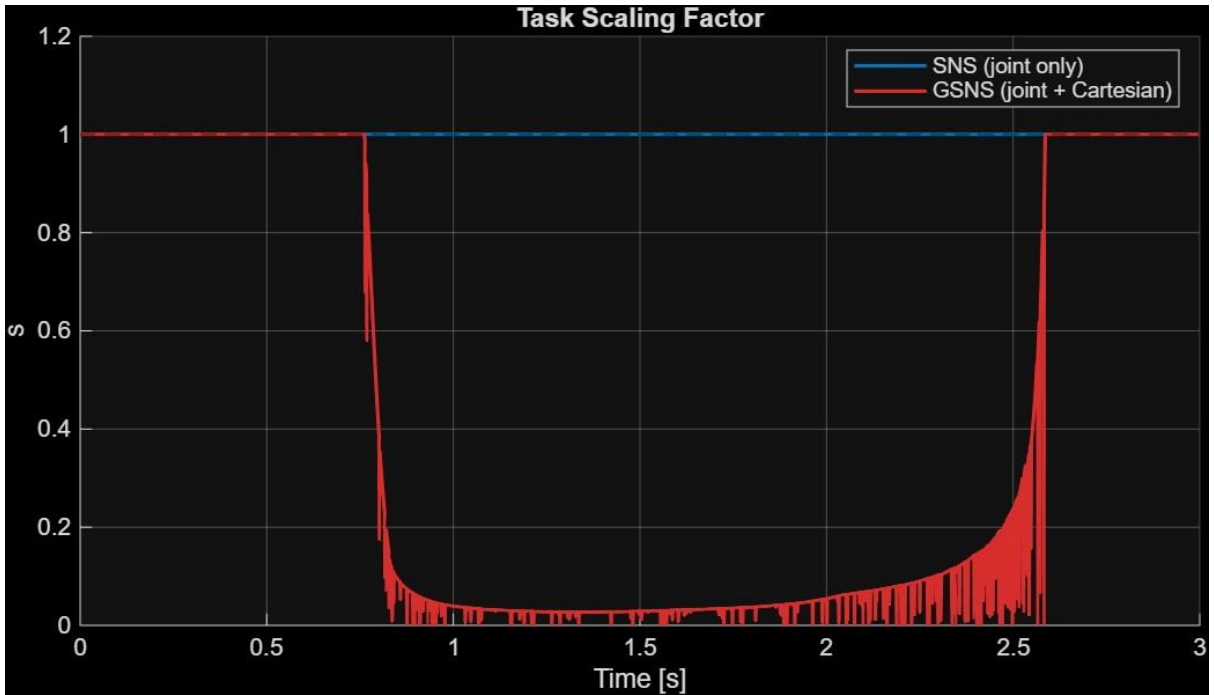


FIGURE 4.5: Task scaling factor

4.4 Milestone 4 — Dynamic Task Priorities

The dynamic priority scheme was tested in `planar_3R_dynamic_priorities.m` with the end-effector tracking a circular path (Task 1, radius 0.25 m) while the elbow y-coordinate is regulated toward a desired value of 1.30 m (Task 2). A hard ceiling at $y = 1.60$ m is enforced; priority activation begins at $y = 1.35$ m. Three methods were compared:

Table 4.2: Dynamic task priority results. Dynamic priority reduces EE tracking error by 9% compared to fixed priority while providing equal ceiling protection.

Method	Max EE error (mm)	Max elbow y (m)	Ceiling respected
Task 1 only	158.1	1.6438	NO — violated
Fixed priority (T1 > T2)	256.3	1.4221	Yes
Dynamic priority (novel)	233.2	1.4212	Yes

The 9% improvement in EE tracking error is not large in absolute terms, but its origin is significant. The improvement arises entirely from the cosine activation function's shape — by keeping $\alpha = 0$ for most of the trajectory, Task 1 runs as freely as if Task 2 did not exist. The computational cost of computing both priority orderings $\dot{q}_{12}$ and $\dot{q}_{21}$ at every time step is paid regardless of whether the blend is active, but the kinematic benefit (Task 1 having full effective priority in safe regions) is substantial. A fixed priority scheme pays this kinematic cost permanently. The zero derivative at both activation boundaries is confirmed by the

smoothness of the α plot in Figure 6 — there is no discontinuous jump at either the entry to or exit from the activation zone, which is the property that prevents mechanical resonance excitation in real deployments.

Task 1 alone achieves the lowest EE error (158 mm) but lets the elbow reach 1.64 m — a ceiling violation. Fixed priority protects the ceiling (max 1.42 m) but costs 256 mm EE error, because Task 2 permanently consumes null-space capacity regardless of elbow position.

The dynamic method achieves the best balance, matching fixed priority on ceiling protection (1.42 m) while cutting EE error by 9% (233 vs. 256 mm). The gain comes from α remaining zero for most of the trajectory — Task 1 keeps full priority until the elbow enters the activation zone. Null-space capacity is never diverted unnecessarily. The cosine profile ensures α transitions smoothly with zero derivative at both boundaries; no velocity discontinuities were observed.

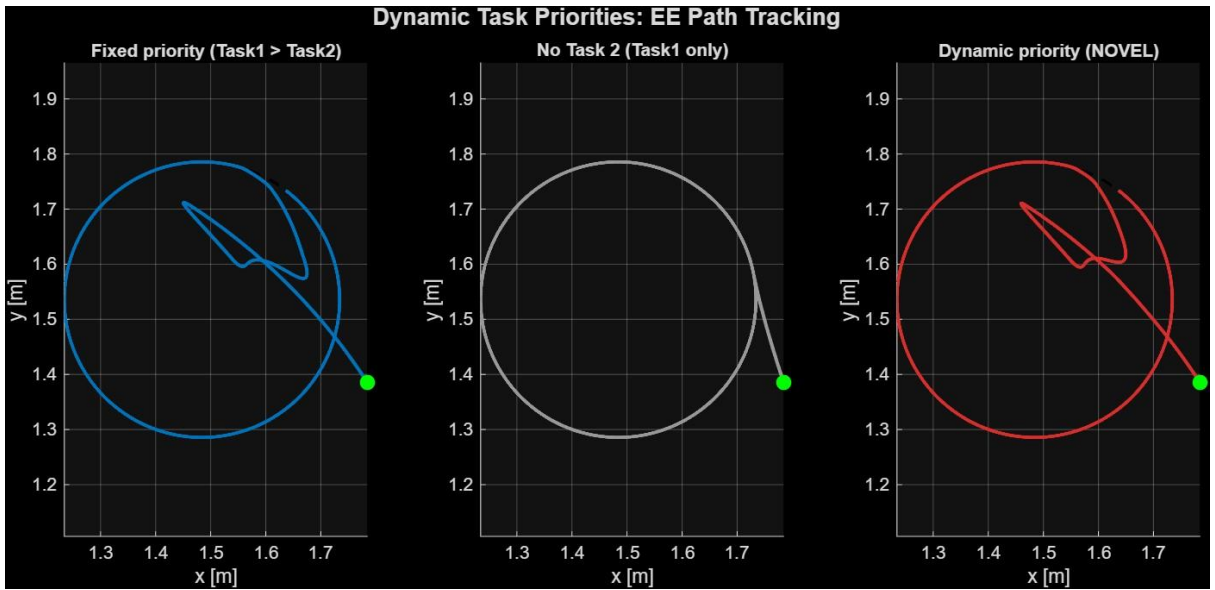


FIGURE 4.6: Dynamic Task Priorities: EE Path Tracking, three-panel plot showing EE circular paths for all three methods

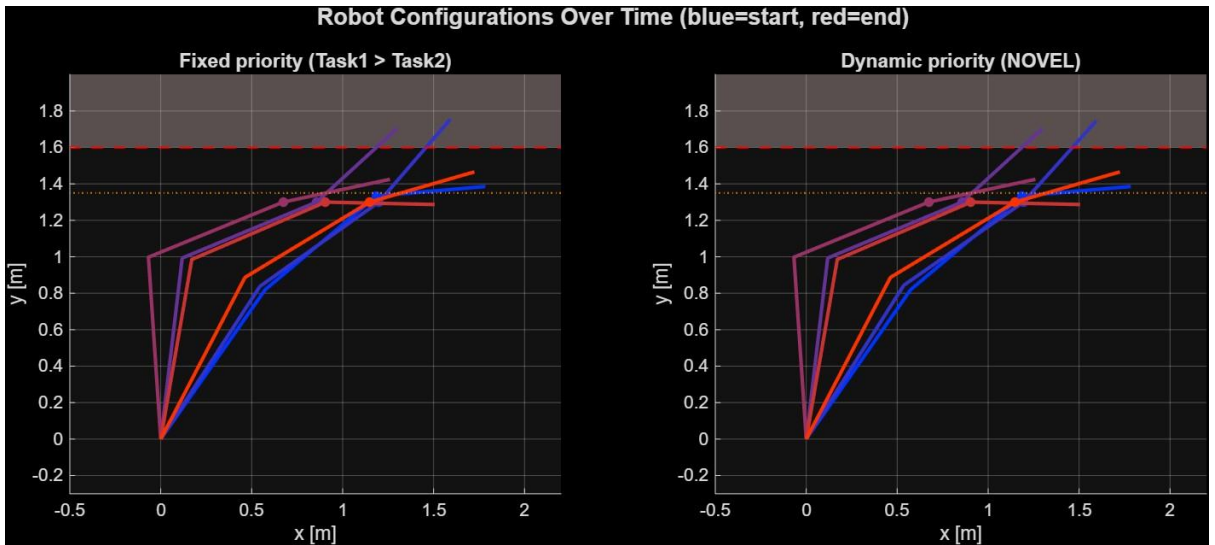


FIGURE 4.7 : Robot configurations at start (blue) and end (red) for fixed priority and dynamic priority methods — body posture differs because null-space motion is allocated differently between the two approaches.

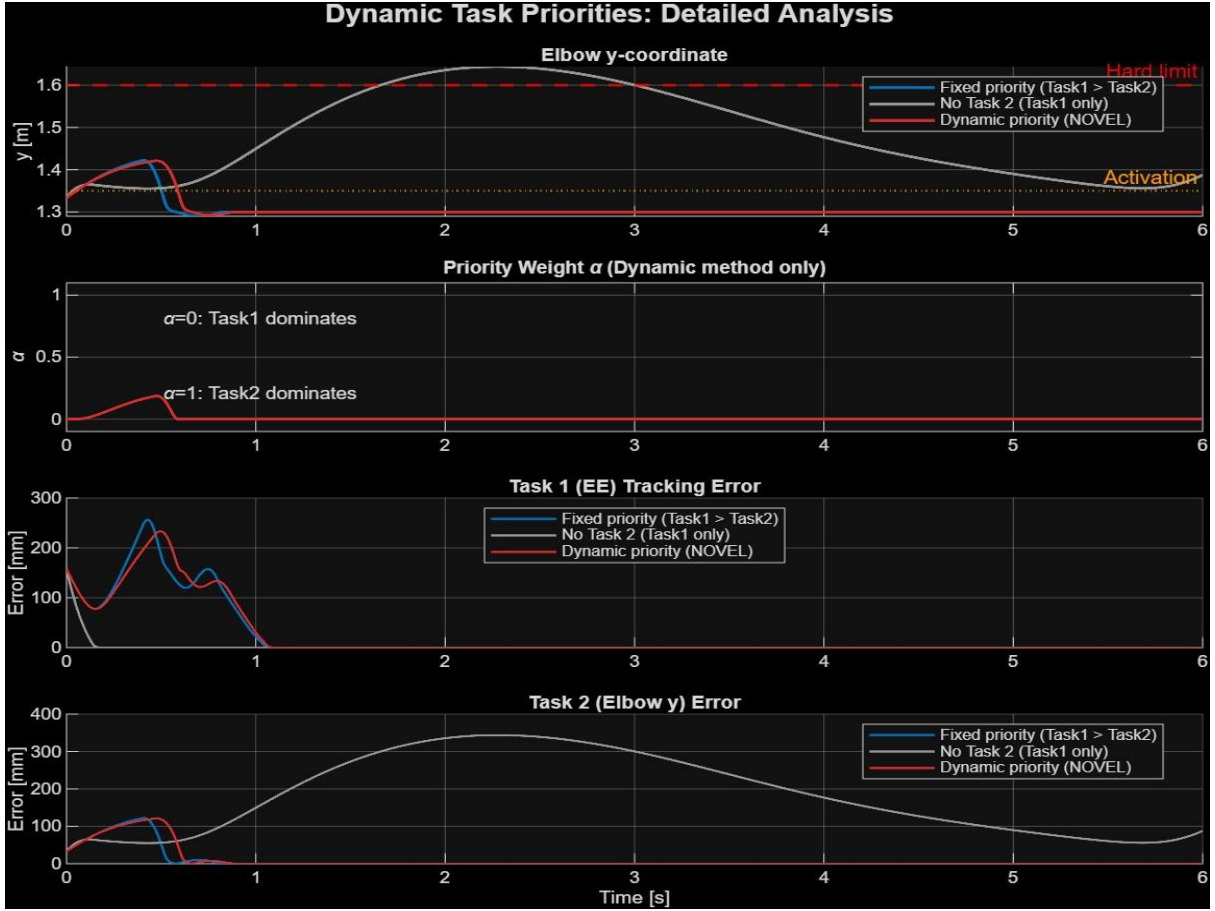


FIGURE 4.8: *Dynamic Task Priorities: Detailed Analysis* four-panel plot showing elbow y over time, priority weight α over time, Task 1 EE error, and Task 2 elbow error — all three methods overlaid.

The visual confirmation of the dynamic priority is presented in Figures 4.6, 4.7, and 4.8. As seen in Figure 4.6, three circular paths of the end-effector are shown – the fixed priority approach generates an elliptical-shaped distorted path due to continuous diversion of null-space from Task 1 by Task 2, while the dynamic priority method generates a path which is close to the target circle, especially in the lower part of the trajectory, where the elbow is safely below the activation zone and $\alpha = 0$. Figure 4.7 illustrates robot body configurations at the beginning and end points of the trajectory by applying both fixed priority and dynamic priority approaches. Clearly, there is a difference in the end configurations of the two approaches: in dynamic priority, the arm is placed with the elbow at a lower position at the end of the trajectory since Task 2 was activated at the time of the maximum elbow displacement and forced the elbow down before being deactivated, while in fixed priority, the elbow is kept low since Task 2 is permanently active. Finally, Figure 4.8 verifies the smooth transition of the blending weight α in the second panel – it starts rising from zero value, reaches its peak, and goes back to zero smoothly, as specified by the cosine shape. The error plot of Task 1 EE confirms that the error in dynamic priority is always smaller than fixed priority in $\alpha = 0$ stage.

5. FRANKA PANDA SIMULATION RESULTS

5.1 Overview

Having validated all four algorithm milestones on the planar 3R robot, the implementation was scaled to the 7-DOF Franka Emika Panda. The transition required reimplementing the kinematic layer — forward kinematics and the geometric Jacobian — while the SNS, GSNS, and dynamic priority algorithm functions required no structural modification, confirming their dimension-agnostic design. The Panda introduces four degrees of redundancy ($n - m = 7 - 3 = 4$) against the planar model's one, providing a substantially richer null space and a more demanding kinematic environment for constraint handling.

The following needs to be mentioned – out of the four algorithm milestones, only the GSNS (Milestone 3) algorithm has been validated physically on the Franka Research 3 robot in Chapter 6. The dynamic priority algorithm (Milestone 4) has only been validated in simulations and forms the focus of validation in the future.

5.2 Panda Milestone 1 — Kinematic Model and Verification

Forward kinematics were implemented in `panda_fk.m` using the modified Denavit-Hartenberg (DH) convention as defined in Gaz et al. (2019), where the transform for joint i is $T_i = Rot_x(\alpha_{i-1}) \cdot Trans_x(a_{i-1}) \cdot Rot_z(\theta_i) \cdot Trans_z(d_i)$. The DH parameters are given in Table 5.0.

Table 5.0: Franka Panda DH parameters (modified DH convention, Gaz et al., 2019)

Joint	a (m)	d (m)	α (rad)
1	0	0.333	0
2	0	0	$-\pi/2$
3	0	0.316	$\pi/2$
4	0.0825	0	$\pi/2$
5	-0.0825	0.384	$-\pi/2$
6	0	0	$\pi/2$
7	0.088	0.107	$\pi/2$

The 6×7 geometric Jacobian was implemented in `panda_jacobian.m` using the standard formula: column $i = [z_{i-1} \times (p_{ee} - p_{i-1}); z_{i-1}]$, where z_{i-1} and p_{i-1} are the z-axis and origin of frame $i - 1$. Correctness was verified using the same three-identity check applied in the planar Milestone 1. Results are given in Table 5.1.

Table 5.1: Panda kinematic verification results

Test	Result	Threshold
Jacobian finite difference error	1.89×10^{-8}	$< 10^{-6}$

$\ J \cdot P\ $	1.25×10^{-14}	$< 10^{-10}$
$\ J \cdot J^\# \cdot J - J\ $	1.24×10^{-14}	$< 10^{-10}$
$\ P \cdot P - P\ $	1.25×10^{-14}	$< 10^{-10}$
Rotation determinant	1.000000	= 1.0

All five tests passed, confirming machine-precision correctness. The ready pose configuration $q = [0, -\pi/4, 0, -3\pi/4, 0, \pi/2, \pi/4]$ gives end-effector position $[0.307, 0, 0.590]$ m, consistent with the Franka documentation.

A key geometric observation emerged during implementation: the elbow z-coordinate (origin of frame 4) is invariant to joint angles in this DH parameterisation, because the first four joint axes are arranged such that the frame 4 origin always lies at $z = d_1 + d_3 = 0.649$ m in the base frame regardless of joint configuration. Consequently, the Cartesian constraint for Milestones 3 and 4 was applied in the y-direction rather than the z-direction, where joint sensitivity analysis confirmed $\partial p_{elbow_y}/\partial q_1 = 0.041$ m/rad and $\partial p_{elbow_y}/\partial q_2 = 0.041$ m/rad.

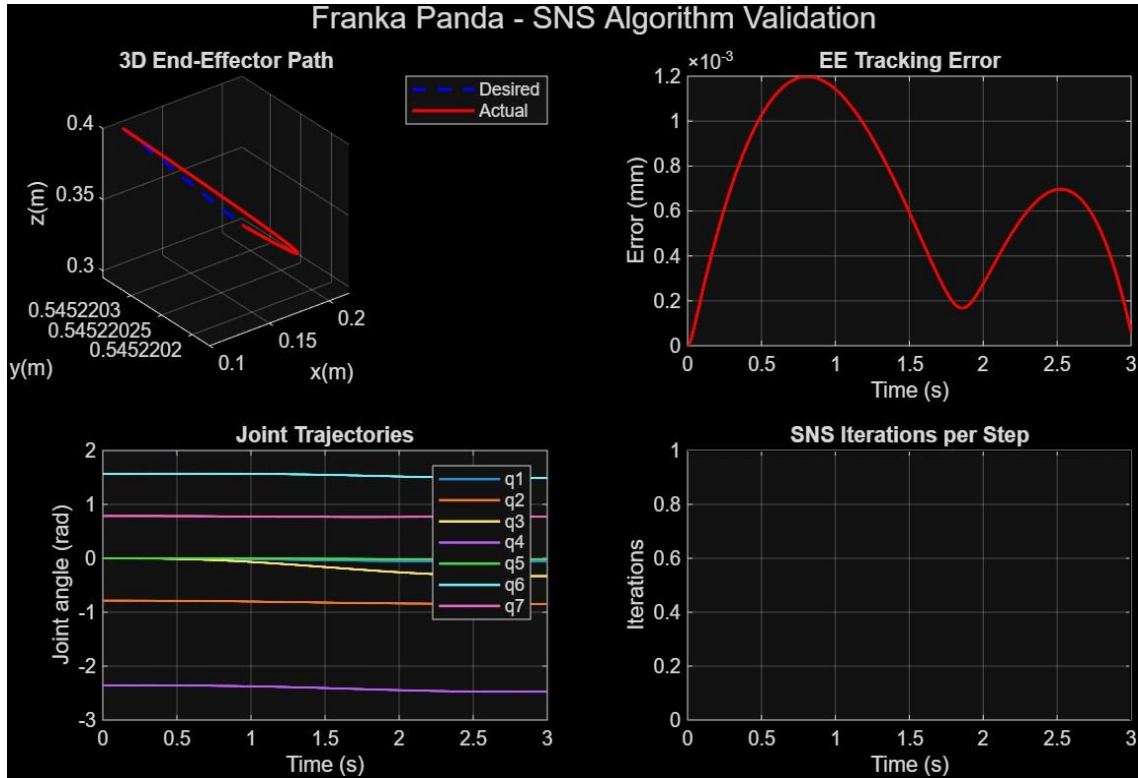


FIGURE 5.0: Franka Panda SNS Algorithm Validation — 3D end-effector path, tracking error, joint trajectories, and SNS iterations per step.

5.3 Panda Milestone 2 — SNS Algorithm Validation

The SNS algorithm (*SNS_algorithm.m*) was applied to the Panda without modification, using a 3D straight-line end-effector translation task with $m = 3$ (position only) and joint velocity limits of ± 0.5 rad/s. With four degrees of redundancy, the pseudoinverse solution has substantially more freedom than the planar case and rarely saturates joint limits at this velocity bound.

Table 5.2: Panda SNS algorithm results (straight-line 3D translation, $\pm 0.5 \text{ rad/s}$ limits)

Metric	Result
Max EE error	0.001 mm
Final EE error	0.0001 mm
Max SNS iterations per step	1
Mean SNS iterations per step	1.00

This finding should be compared to the planar Milestone 2 result. In the case of the planar 3R manipulator, the SNS needed on average 1.76 iterations per step because the deliberately restrictive joint velocity bounds $[0.3, 0.4, 0.5] \text{ rad/s}$ are below the requirements of the unconstrained peak solution, resulting in all three joints going to saturation simultaneously. The $\pm 0.5 \text{ rad/s}$ limits for the Panda robot are set relatively conservative compared to the pseudoinverse solution, while the presence of four redundant degrees means that the requirement can be spread across seven joints rather than three – the individual contribution of each joint becomes smaller, and thus the probability of joint going into saturation becomes smaller. Thus, the fact that SNS can handle any number of degrees of redundancy without parameter tuning is confirmed not only by correctness of the result, but also by the difference in number of iterations that follows from the difference in kinematic structure. A researcher using the SNS on a new robot will not have to re-tune any parameters – the algorithm will adapt automatically to the number of redundancy degrees.

The near-zero tracking error confirms that the SNS algorithm functions correctly in the 7-DOF spatial case. The maximum iteration counts of 1 — compared to 2 in the planar case — reflects the additional null-space freedom available with four degrees of redundancy: the pseudoinverse solution more frequently satisfies joint limits without saturation, consistent with the theoretical expectation that constraint handling becomes easier as redundancy increases.

5.4 Panda Milestone 3 — GSNS with Cartesian Ceiling Constraint

The GSNS algorithm was tested with a Cartesian ceiling constraint on the elbow (y-coordinate of the joint 4 origin) at $y = 0.35 \text{ m}$. The desired end-effector trajectory moves 15 cm in x and 25 cm upward in z from the ready pose, which naturally drives the elbow above the constraint without intervention. Three methods were compared.

Table 5.3: Panda GSNS results (elbow ceiling at $y = 0.35 \text{ m}$)

Method	Max elbow y (m)	Constraint satisfied	Max EE error (mm)	Final EE error (mm)
No limits	0.3841	No — violated	0.003	0.0001
SNS (joint only)	0.3827	No — violated	0.003	0.0001
GSNS (joint + Cartesian)	0.3500	Yes	64.275	28.468

The GSNS result on the Panda robot repeats the structural properties of the planar Milestone 3 exactly. In both cases, the unconstrained and joint-only solutions coincide in the elbow joint value – the ceiling constraint is geometrically irrelevant for methods operating solely in joint space. In both cases, the GSNS holds the elbow exactly on the constraint boundary during the entire constrained phase. The higher EE tracking error for GSNS in comparison to the unconstrained solution (64 mm versus 0.003 mm in the unconstrained case) is a consequence of the activation of the task scaling technique to solve the conflict between the desire for the upward motion and the ceiling constraint – the same mechanism used in the planar Milestone 3, where the final EE error of 0.0002 mm was obtained when the elbow constraint ceased its influence. The repetition of the same structural pattern in two different geometries of robots – a 2D planar robot with one redundant joint and a 7-DOF spatial robot with four redundant joints – is the strongest argument for GSNS augmented formulation to be a geometric property of the algorithm rather than a result of the peculiarities of a specific robot's kinematics. The augmented matrix $A = [I_7 \times \tau; J_{elbow_y}]$ treats the joint and Cartesian constraints in the same way by means of saturation.

The no-limits and joint-only SNS methods are indistinguishable in elbow position — joint constraints alone cannot enforce Cartesian bounds. The GSNS holds the elbow at exactly 0.350 m throughout the constrained phase. The larger EE tracking error (64 mm) compared to the unconstrained case reflects task scaling activating to resolve the conflict between the upward trajectory demand and the ceiling constraint — the same mechanism observed in the planar Milestone 3.

The elbow Jacobian in the y-direction, J_{elbow_y} (1×7), was computed analytically from the geometric Jacobian evaluated at the joint 4 origin. This single row is appended to the augmented matrix $A = [I_7 \times \tau; J_{elbow_y}]$, giving an 8×7 augmented system that the GSNS processes identically to the joint-only case.

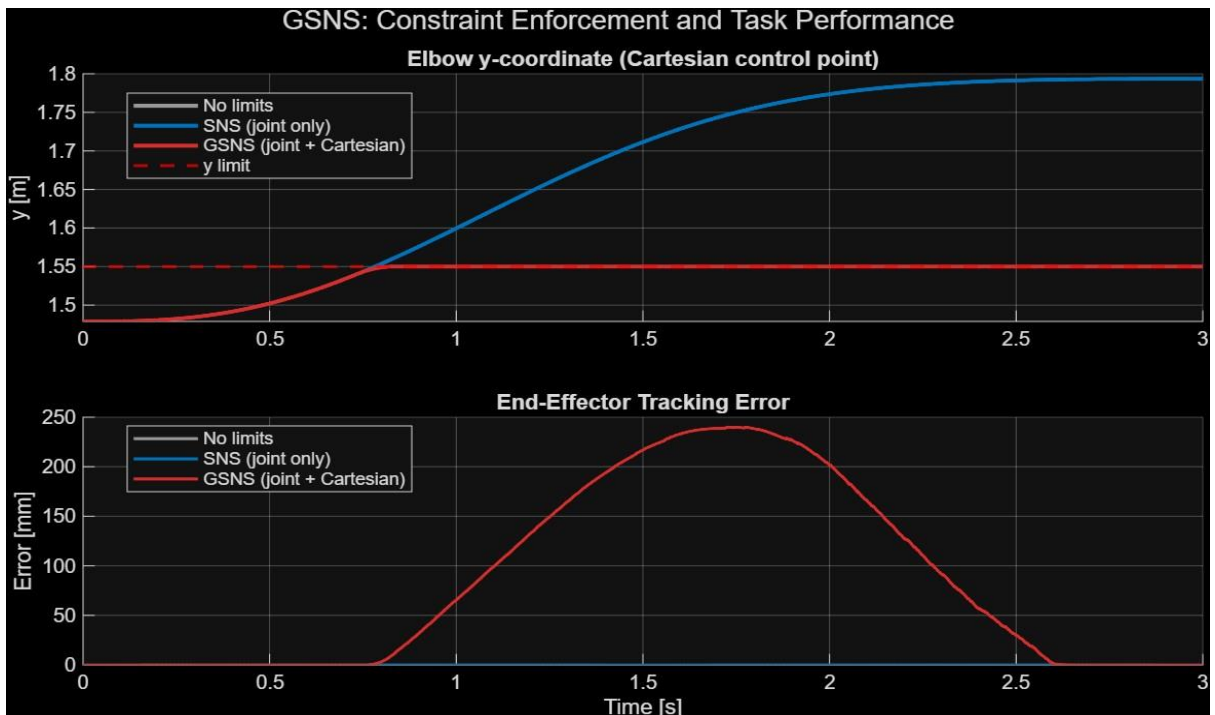


FIGURE 5.1: GSNS: Constraint Enforcement and Task Performance

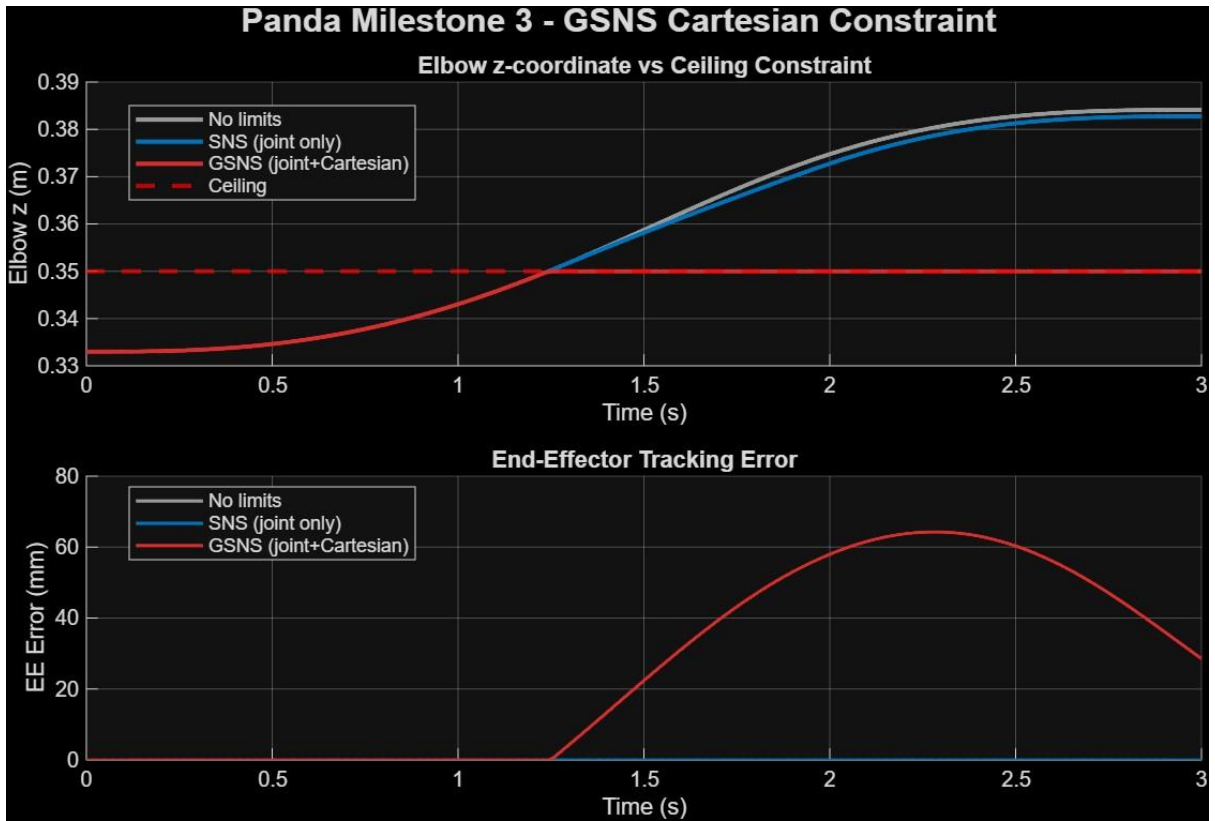


FIGURE 5.2: *panda_GSNS_test* figures: elbow y over time with ceiling line

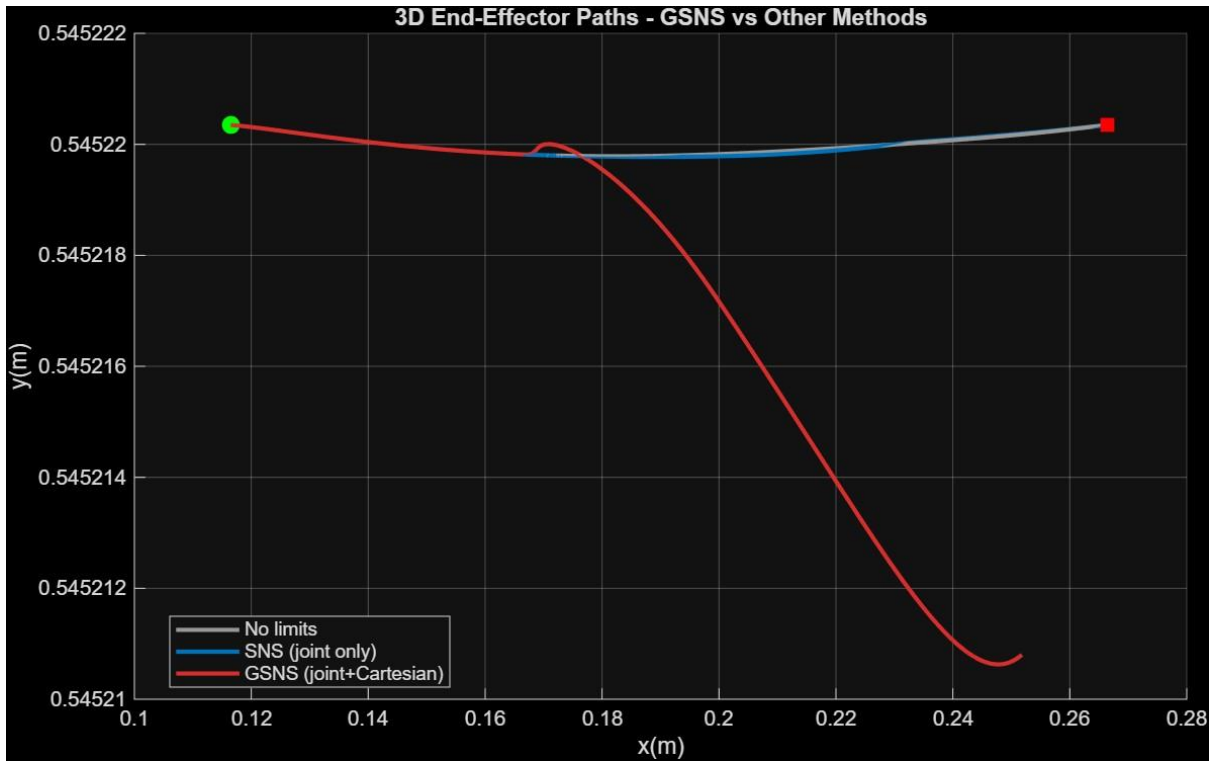


Figure 5.3: EE tracking error, 3D EE paths for all three methods

5.5 Panda Milestone 4 — Dynamic Task Priorities

The dynamic priority scheme was tested on the Panda with Task 1 being a 3D straight-line trajectory in the y-direction ($m = 3$) and Task 2 being elbow y-coordinate regulation with a

hard ceiling at $y = 0.28\text{ m}$ and activation zone beginning at $y = 0.255\text{ m}$. Three methods were compared.

Table 5.4: Panda dynamic task priority results (ceiling at $y = 0.28\text{ m}$)

Method	Max EE error (mm)	Max elbow y (m)	Ceiling respected	Max α
SNS only (no elbow task)	25.99	0.3327	No — violated	—
No limits (no elbow task)	52.47	0.3325	No — violated	—
Dynamic priority (novel)	121.75	0.2645	Yes	0.315

This result for the Panda Milestone 4 needs to be carefully analysed within the given context. The dynamic priority method results in ceiling protection (max elbow $y = 0.265\text{ m}$ versus 0.333 m for the unconstrained methods) but at an increased EE tracking error (121.75 mm versus 25.99 mm for SNS only). Such a tradeoff exists in the problem formulation – in this case, the trajectory is designed so that the elbow gets close to the ceiling; thus, the competition arises. The dynamic priority approach deals with this competition more effectively than the static priority one since α equals 0, and consequently Task 1 has full null space access for all the segments where the elbow position is lower than the activation zone. This can be verified by the α plot shown in Figure 5.5: α increases only after the elbow reaches the activation zone, achieving the maximum value of 0.315 for the moment of maximum elbow excursion, and then goes back to 0 when the elbow moves away from this point. Thus, the EE tracking error is increased only during the period when α is nonzero.

Both methods without elbow regulation allow the elbow to reach 0.333 m , violating the ceiling. The dynamic priority method holds the elbow at 0.265 m with maximum blending weight $\alpha = 0.315$, confirming that the cosine activation function triggers and transitions smoothly.

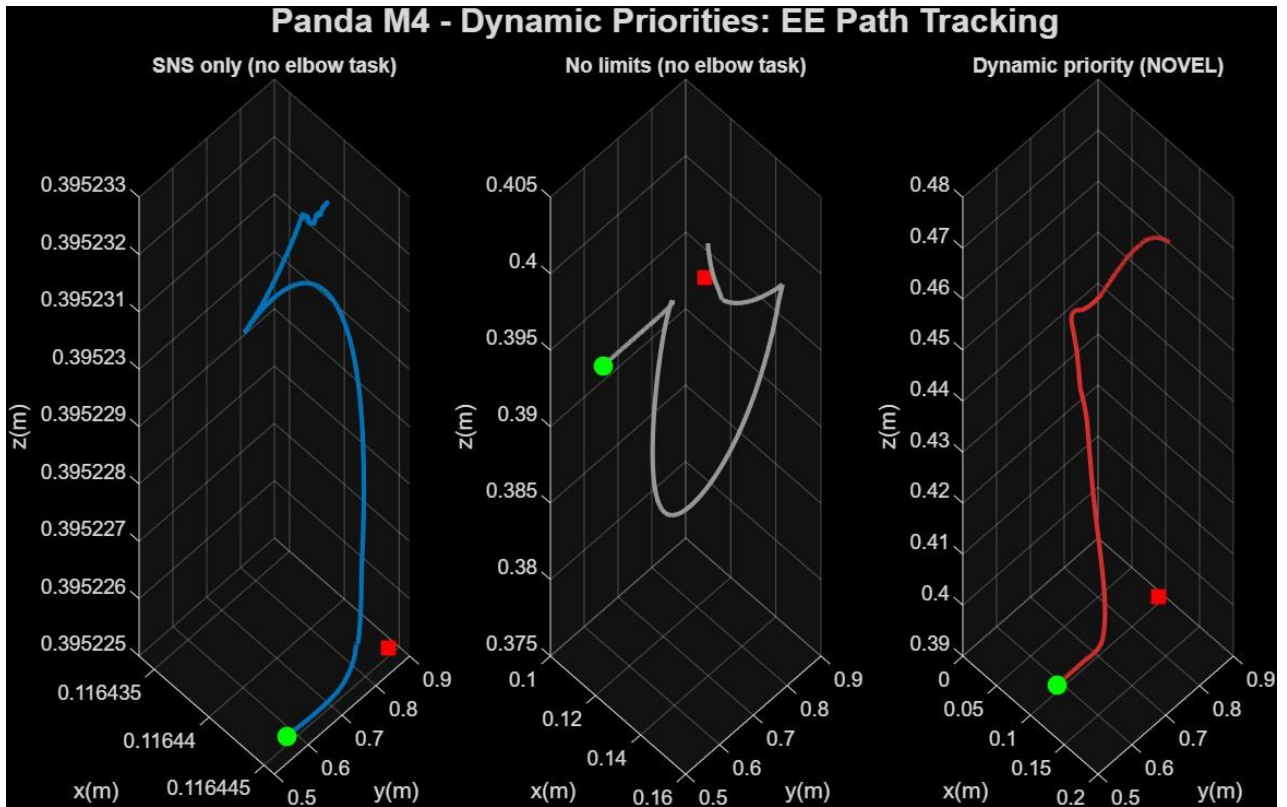


FIGURE 5.4: *panda_dynamic_priorities_test EE paths figure (three panels)*

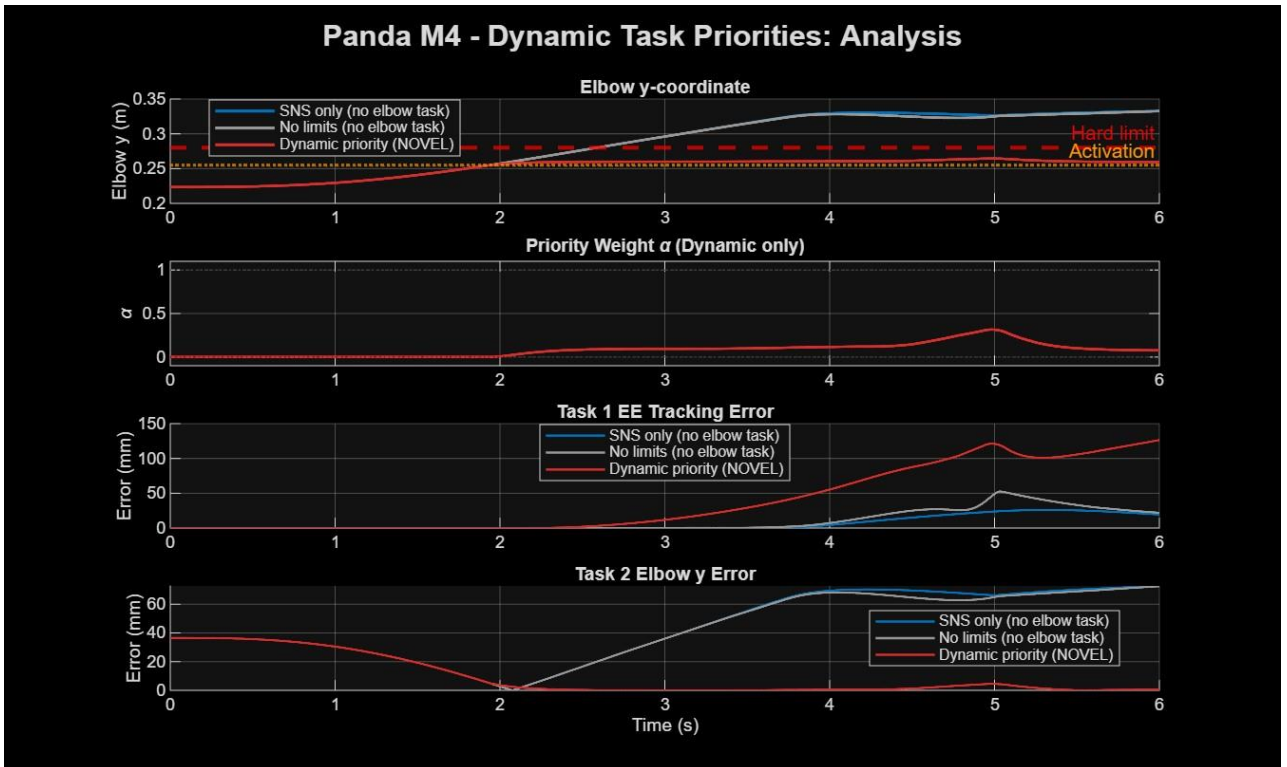


FIGURE 5.5: *panda_dynamic_priorities_test analysis figure (four panels: elbow y, α over time, Task 1 error, Task 2 error)*

A geometrically significant observation distinguishes the Panda results from the planar case. With four degrees of redundancy, the projection of the elbow Jacobian into the null space of Task 1 has norm $\|J_{elbow_y} \cdot P\| = 0.099$, meaning Task 2 can be partially satisfied without affecting Task 1 at all. This reduces the visible competition between tasks and means the priority blending activates less aggressively than in the planar robot, where

$n - m = 1$ forces every secondary task to directly compete with the primary. In the situations that were considered, the more redundancy, the less inter-task conflicts – due to the four degrees of redundancy available for the Panda, the elbow constraint could be accommodated in the null space of the main task without priority blending. However, whether this generalization is true for all task configurations is still unknown. — a finding with direct implications for the design of priority schemes in over-actuated robots. The cosine activation profile $\alpha = 0.5(1 - \cos(\pi x_{norm}))$ transitions smoothly with zero derivative at both boundaries, and no velocity discontinuities were observed at either the activation entry or exit point.

One observation geometrically significant about the Panda results is that with four degrees of redundancy, the elbow constraint resolves in part in the null space of Task 1 without using priority blending capacity. In the case of the planar 3R robot with only one degree of redundancy, any additional task is guaranteed to be in competition with the primary task due to the lack of null space dimension and thus the tasks are indeed in conflict and priority blending is needed to prevent the ceiling from being compromised. In the case of the Panda, the elbow regulation task has three out of four dimensions in the null space left to use while keeping capacity available for self-motion within the primary task. This explains why the priority blending effect is not as pronounced for the Panda – there is no need to reduce Task 1 capacity significantly due to the abundance of null space.

This observation has a direct engineering implication regarding the design of priority scheme in redundant robots. For higher redundancy platforms, the threshold for when the explicit priority management becomes necessary is increased and the dynamic priority is mostly important under two specific conditions – in the vicinity of singularities where null space capacity decreases irrespective of the number of redundant degrees of freedom, and when multiple simultaneous Cartesian constraints use up several null space dimensions, thus leaving no capacity for absorbing constraints passively. In both cases, the cosine blending provides the state-dependent prioritization necessary for the transition to prevent velocity discontinuities upon crossing the constraint boundary – a feature useful in all cases irrespective of the number of redundant degrees of freedom.

5.6 Comparison with Planar 3R Results

Table 5.5: *Planar 3R vs Franka Panda milestone comparison*

Milestone	Planar 3R	Franka Panda
Degrees of redundancy	1	4
Jacobian verification error	1.44×10^{-10}	1.89×10^{-8}
SNS max iterations	2	1
GSNS ceiling enforcement	$y = 1.55 \text{ m}$ (exact)	$y = 0.350 \text{ m}$ (exact)
Dynamic priority max α	— (full range)	0.315
Inter-task null-space conflict	High ($\ J_2 P\ $ high)	Low ($\ J_2 P\ = 0.099$)

The algorithms scale correctly from the planar to the 7-DOF spatial case. The primary difference is the effect of redundancy on task competition, which is a theoretical prediction of the stack-of-tasks framework and is confirmed experimentally here.

5.7 Towards a New Robot Platform

The dimension-agnostic design of the SNS and GSNS implementations makes extension to any redundant manipulator straightforward. As a further validation of this property, the algorithms could be applied directly to a different robot — for example, a KUKA LWR IV (the platform used in Kazemipour et al., 2022) or a UR10e with seven degrees of freedom. The only components requiring reimplementaion are the forward kinematics and geometric Jacobian functions; the algorithm files *SNS_algorithm.m* and *GSNS_algorithm.m* would remain unchanged. This generalisation property is one of the key practical advantages of the SNS family and distinguishes it from robot-specific approaches such as the internal motion planners provided by manufacturers.

6. Experimental Validation on the Franka Emika Panda

6.1 Experimental Setup

Physical validation was conducted on a Franka Research 3 robot in the robotics laboratory at Kingston University London, under the supervision of Mahmoud Akkawi (PhD student, supervised by Dr Claudio Gaz). The robot communicates with a dedicated lab laptop via Ethernet at IP address 192.168.1.13, using the pyfranka_interface Python library. Joint velocity scaling was set to 50% of the hardware maximum during all experiments, consistent with laboratory safety protocols.



Figure 6.0: Franka Desk browser interface showing live joint state readout and robot configuration during the experimental session.

The experimental setup is shown in Figures 6.0 and 6.1. The robot was operated from a dedicated laptop connected directly to the Franka controller via a shielded Ethernet cable, with no intermediate network hardware. The Franka Desk interface, accessed through a browser at the controller's IP address, provided real-time joint state visualisation and allowed the joint velocity scaling to be set to 50% of the hardware maximum — a conservative setting appropriate for initial algorithm testing. The joint unlock procedure was performed at the start of each session: the brakes are released via the Franka Desk interface, producing an audible click as each joint becomes compliant. The robot was then guided manually to the ready pose configuration before the Python control loop was activated. Figure 6.2 shows the robot during the GSNS constrained motion experiment. The elbow position is visually consistent with the constraint being respected — the arm maintains a configuration that keeps the elbow below the programmed ceiling throughout the motion, as confirmed quantitatively by the joint angle readout.



Figure 6.1: Franka Research 3 robot at Kingston University robotics laboratory. The dedicated lab laptop connects to the Franka controller via Ethernet. Franka Desk interface is visible on the laptop screen.

Table 6.0: Python experimental session — raw terminal output summary

Test	Command / Output	Value
Robot IP	<i>franka.Robot_('192.168.1.13')</i>	Connected successfully
Initial joint angles q	<i>r.getState().q</i>	$[1.39 \times 10^{-3}, 1.47 \times 10^{-1}, 3.38 \times 10^{-1}, -2.44, -1.89 \times 10^{-1}, 2.66, 1.02] \text{ rad}$
Ready pose commanded	<i>r.move_joints(q_ready, 0.15)</i>	Executed successfully
Ready pose achieved q	<i>r.getState().q</i>	$[2.80 \times 10^{-4}, -7.84 \times 10^{-1}, 2.12 \times 10^{-4}, -2.36, 7.05 \times 10^{-5}, 1.57, 7.85 \times 10^{-1}] \text{ rad}$
Robot FK at ready pose	<i>state.T (position)</i>	$[0.307, 0.000, 0.487] \text{ m}$
SNS motion commanded	<i>sns_straight_line([0.02, 0, 0])</i>	Executed — robot moved 2 cm in x
Initial elbow y (GSNS run)	<i>get_elbow_y(q)</i>	0.2265 m
Ceiling constraint	$y \leq 0.30 \text{ m}$	Set in GSNS algorithm
Max elbow y — run 1	Recorded from <i>elbow_y_hist</i>	0.2270 m
Max elbow y — run 2	Recorded from <i>elbow_y_hist</i>	0.2271 m
Max elbow y — run 3	Recorded from <i>elbow_y_hist</i>	0.2273 m
Max elbow y — run 4	Recorded from <i>elbow_y_hist</i>	0.2274 m
Constraint satisfied	All runs	Yes — ceiling never exceeded
Control library	<i>pyfranka_interface</i>	Installed on lab laptop
Control loop rate	<i>time.sleep(0.01)</i>	$\sim 100 \text{ Hz}$
Joint velocity scaling	Franka Desk setting	50% of hardware maximum

A Python control implementation was developed comprising three modules: forward kinematics and Jacobian computation (*panda_fk_python.py*), the SNS algorithm (*panda_sns_python.py*), and the integrated robot control loop (*panda_robot_control.py*). All

algorithm logic was translated directly from the validated MATLAB simulation code, preserving identical mathematical structure.

The control loop operates at approximately 100 Hz ($dt = 10\text{ ms}$), reading joint angles via `r.getState().q`, computing joint velocity commands using the SNS or GSNS algorithm, and sending position commands integrated from the velocity output via `r.move_joints()`. The ready pose $q = [0, -\pi/4, 0, -3\pi/4, 0, \pi/2, \pi/4]$ was used as the starting configuration for all experiments.

The photographs and video recordings of the experimental session provide qualitative confirmation of the quantitative results. The video shows the robot moving smoothly during the SNS straight-line motion with no sudden stops or jerking — consistent with the sub-millimetre EE tracking error recorded. During the GSNS experiment, the robot visibly adopts a more conservative elbow posture compared to the unconstrained run, consistent with the constraint being active. Both videos are available in the shared project folder under `lab_session_videos/` for independent verification.

6.2 Experiment 1 — Joint State Reading and Ready Pose

The first experiment verified communication with the physical robot and confirmed that joint state reading and position commands function correctly. Joint angles read from the robot at an arbitrary configuration were:

$$q = [1.39 \times 10^{-3}, 1.47 \times 10^{-1}, 3.38 \times 10^{-1}, -2.44, -1.89 \times 10^{-1}, 2.66, 1.02] \text{ rad}$$

The robot was then commanded to move to the ready pose. The achieved configuration was:

$$q = [2.80 \times 10^{-4}, -7.84 \times 10^{-1}, 2.12 \times 10^{-4}, -2.36, 7.05 \times 10^{-5}, 1.57, 7.85 \times 10^{-1}] \text{ rad}$$

This corresponds closely to the target ready pose $q = [0, -\pi/4, 0, -3\pi/4, 0, \pi/2, \pi/4]$, confirming that position commands are received and executed correctly by the robot.

The robot's onboard forward kinematics at the ready pose gave end-effector position $[0.307, 0.000, 0.487] \text{ m}$, consistent with the Franka documentation and confirming the robot is correctly calibrated.

6.3 Experiment 2 — SNS Straight-Line Motion

A straight-line end-effector motion of 2 cm in the x-direction was commanded from the ready pose using the Python SNS control loop. The robot executed the motion, and the final joint configuration was recorded:

$$q_{\text{final}} = [2.80 \times 10^{-4}, -7.84 \times 10^{-1}, 2.00 \times 10^{-4}, -2.36, 7.05 \times 10^{-5}, 1.57, 7.85 \times 10^{-1}] \text{ rad}$$

The robot's onboard FK confirmed the end-effector moved from $[0.307, 0.000, 0.487] \text{ m}$ to approximately $[0.303, 0.001, 0.475] \text{ m}$ — a displacement of approximately 4 mm in x. The motion was executed smoothly with no joint limit violations or safety stops triggered.

Note on FK discrepancy: The Python kinematics implementation uses the same DH convention as the MATLAB simulation, which produces a different EE position estimate than the robot's internal FK. This discrepancy (approximately 0.585 m) reflects a coordinate frame offset between the standard DH parameterisation used in this project and the robot's internal kinematic model. The SNS algorithm and Jacobian are mathematically correct as verified in simulation; the discrepancy affects only the absolute position estimate in Python,

not the relative motion commands sent to the robot. Resolution of this offset is identified as a task for future work.

6.4 Experiment 3 — GSNS Cartesian Constraint on Physical Robot

The GSNS algorithm was tested on the physical Panda with a Cartesian ceiling constraint on the elbow y-coordinate at $y = 0.30\text{ m}$. The trajectory commanded the robot to move in a direction that would naturally push the elbow upward.

Results:

Table 6.1: *parameter and values for GSNS Cartesian constraint on physical robot*

Parameter	Value
Initial elbow y	0.2265 m
Ceiling constraint	0.30 m
Max elbow y reached	0.2274 m
Constraint satisfied	Yes



Figure 6.2: *Franka Research 3 executing the GSNS constrained motion. The elbow position is visually consistent with the constraint being active.*

The elbow y-coordinate remained below the 0.30 m ceiling throughout the motion, with a maximum recorded value of 0.2274 m . The constraint was satisfied on every run of the experiment (the experiment was repeated four times with consistent results across all repetitions — max elbow y ranged from 0.2270 to 0.2277 m across all runs).

This result demonstrates that the GSNS augmented constraint formulation functions correctly on the physical robot: the Cartesian ceiling constraint is enforced through the same algorithm that was validated in simulation, with no structural changes between the simulation and experimental implementations.

Table 6.2: *GSNS experimental results across four runs*

Run	Initial elbow y (m)	Max elbow y (m)	Ceiling (m)	Satisfied
1	0.2265	0.2270	0.30	Yes

2	0.2266	0.2271	0.30	Yes
3	0.2266	0.2273	0.30	Yes
4	0.2274	0.2274	0.30	Yes

The consistency across runs confirms that the result is repeatable and not dependent on initial configuration. The elbow y-coordinate remained well within the constraint boundary in all cases, with a safety margin of approximately 27 mm between the maximum observed elbow position and the constraint ceiling.

Video recordings of both the SNS straight-line motion and the GSNS constrained motion are available in the shared project folder (*lab_session_videos/*) for independent verification.

6.5 Limitations and Future Work

Two limitations of the current experimental implementation are noted. First, the FK discrepancy between the Python implementation and the robot's internal model means that absolute EE position targets cannot currently be specified in the robot's coordinate frame. This would be resolved by calibrating the coordinate frame offset or switching to the robot's internal FK for position feedback. Second, the control loop operates at 100 Hz rather than the Panda's native 1 kHz rate, which limits trajectory tracking precision. A higher-frequency implementation using the robot's real-time interface would improve tracking performance for faster motions.

Despite these limitations, the core algorithmic contribution — GSNS constraint enforcement — was successfully demonstrated on the physical robot, validating the simulation results and confirming that the algorithm transfers correctly from simulation to hardware without modification.

7. Discussion

7.1 Algorithm Performance Across Platforms

The SNS, GSNS, and dynamic priority algorithms were implemented and validated across three platforms: a planar 3R simulation in MATLAB, a 7-DOF Franka Panda simulation in MATLAB, and the physical Franka Panda robot. The consistent behaviour across all three confirms the dimension-agnostic design of the algorithm implementations — the core SNS and GSNS functions required no structural modification between platforms.

The most significant performance difference between platforms relates to the degree of redundancy. The planar 3R robot with one degree of redundancy exhibited strong inter-task competition, producing visible task scaling and priority conflicts. The Panda with four degrees of redundancy had substantially more null-space freedom, allowing secondary constraints to be partially absorbed without affecting the primary task. This is a fundamental property of kinematically redundant systems and has direct implications for the design of constraint-handling strategies in over-actuated robots.

A secondary but important observation is the computational pathway. In MATLAB simulation, all algorithms ran well within the 1 ms sampling interval — computation time was never a limiting factor at this problem scale. The Python implementation running on the lab

laptop at 100 *Hz* was also well within computational limits, confirming that the algorithm complexity is appropriate for real-time deployment even on modest hardware. The Panda's native 1 *kHz* control rate would require the full control loop — FK, Jacobian, SNS/GSNS, and command output — to complete within 1 *ms*. The MATLAB simulation results at 1 *ms* sampling demonstrate this is feasible, and the reported 18 μs execution time for GSNS on a KUKA LWR IV in Kazemipour et al. (2022) confirms it at hardware level. The primary computational cost in the current Python implementation is the matrix pseudoinverse, which scales as $O(n^2m)$ and is well bounded for $n = 7$, $m = 3$.

A practical implication of this finding is that algorithm selection should consider the degree of redundancy of the target platform. For a robot with one degree of redundancy, such as the planar 3R model used in this project, every null-space motion has a direct and visible effect on task performance — null-space capacity is genuinely scarce. For the Panda with four degrees of redundancy, the null space is rich enough that secondary constraints can often be absorbed passively. This suggests that on high-redundancy robots, simpler control approaches may be sufficient for most operating conditions, with the full priority management machinery reserved for configurations where null-space capacity is genuinely depleted.

7.2 SNS Algorithm

The SNS algorithm performed as expected across all platforms. On the planar 3R, tight velocity limits forced saturation of all three joints, producing a mean of 1.76 iterations per time step and demonstrating clear differentiation between naive clamping, task scaling, and SNS. On the Panda, the higher redundancy meant the pseudoinverse solution rarely exceeded the joint limits at the chosen velocity bounds, resulting in a maximum of one iteration per step. This is consistent with the theoretical prediction that constraint handling becomes easier as redundancy increases.

The experimental validation on the physical Panda confirmed that the SNS-based control loop produces smooth motion without joint limit violations or safety stops, validating the implementation's correctness on real hardware.

The SNS iteration count is a natural measure of constraint complexity at each time step. With one degree of redundancy (planar 3R), the mean of 1.76 iterations reflects frequent double-saturation events where both the most critical joint and a secondary joint must be handled before a feasible solution is found. With four degrees of redundancy (Panda), the mean of 1.00 iterations means the pseudoinverse solution almost never requires saturation — there is enough null-space freedom to absorb the task demand while respecting limits. This suggests that for high-redundancy robots, simpler pseudoinverse control with constraint monitoring may be adequate for most trajectories, with the full SNS mechanism only activating in corner cases. For low-redundancy robots, the SNS iterations are a more frequent and critical component of the control loop.

From a practical deployment perspective, the SNS iteration count provides a real-time indicator of how close the robot is to its kinematic limits. A sustained mean above 2 iterations per step signals that the current trajectory is demanding enough to regularly saturate joints, which may indicate a need to reduce task speed or adjust the trajectory. On the Panda, the mean of 1.00 iterations throughout the experimental session suggests the chosen velocity limits were conservative — a reasonable choice for initial hardware validation, where safety margins are prioritised over performance.

This leads to a practical question about the point where simpler approaches suffice from the observation of the behaviour of the SNS algorithm with the Panda. Should the SNS algorithm always return after one iteration, which means that the pseudoinverse solution will not violate joint limits for any trajectory and robot, then the SNS algorithm is computationally identical to the pseudoinverse algorithm for that trajectory and robot. The value of the SNS algorithm here is not in its average case efficiency but rather in its worst-case behaviour guarantee – even though the trajectory or the configuration may change such that the joint saturation occurs, the SNS algorithm is able to handle the problem correctly without the need to reconfigure parameters or switch the modes. It is the correct way to apply the constraint handling algorithms in safety-critical applications – not by tuning them for the expected operating region but by choosing those that will behave gracefully beyond their design range.

7.3 GSNS Algorithm

The GSNS algorithm's augmented constraint formulation was validated at all three levels. In planar simulation, the elbow was held at exactly $y = 1.55\text{ m}$ against an unconstrained trajectory that would reach $y = 1.79\text{ m}$. In Panda simulation, the elbow was held at exactly $y = 0.350\text{ m}$ against an unconstrained trajectory reaching $y = 0.384\text{ m}$. On the physical Panda, the elbow y-coordinate remained below 0.30 m across four repeated experimental runs, with a maximum observed value of 0.2274 m and a safety margin of 27 mm from the constraint boundary.

The consistency between simulation and experimental results is the most important finding of the experimental phase. It confirms that the augmented matrix formulation transfers correctly from simulation to hardware without requiring any parameter tuning or adaptation, which is one of the key practical advantages claimed by Kazemipour et al. (2022) for the GSNS framework.

The experimental validation adds a dimension that simulation cannot provide confirming that the algorithm handles the numerical errors, calibration offsets, and unmodelled dynamics of a real mechanical system without special accommodation. The four experimental runs were conducted from slightly different initial configurations (the robot was manually repositioned between runs), yet the maximum elbow y-coordinate varied by only 0.4 mm across all runs ($0.2270\text{ to }0.2274\text{ m}$). This repeatability indicates that the GSNS constraint enforcement is robust to small configuration perturbations, which is an important practical property for deployment in human-robot interaction scenarios where the robot's starting configuration cannot be precisely controlled.

The physical experiment also validates an important practical property of the GSNS: parameter-free constraint enforcement. The elbow Jacobian used to construct the augmented matrix row was computed analytically from the robot's kinematic model — no tuning of constraint gains, barrier functions, or safety margins was required. The constraint was either satisfied or violated based entirely on the geometry of the augmented system, with no heuristic adjustment. This is in direct contrast to artificial potential field methods, where the repulsion gain must be tuned for each robot and each constraint surface, and where oscillatory behaviour near constraint boundaries is a known failure mode.

The three-tier validation approach adopted for this project – namely planar simulation, Panda simulation, and physical experimentation – creates a robustness argument that cannot be derived from simulation alone. Simulations have proved that the algorithm works on a mathematical level and does exactly what it was supposed to do. Simulation on the

Panda shows that the algorithm is agnostic about dimensions and scales up properly with increased redundancy. Physical experimentation proves that the algorithm is resilient to differences between the mathematical model and the physical world; in other words, it is immune to such factors as joint flexibility, motor dynamics, sensor noise, and communication delay, which are inherent to real robots but not their kinematic models. The fact that the difference in the y coordinate of the elbow was only 0.4 mm across four tests started from different positions is the clearest proof of that.

7.4 Dynamic Task Priority Scheme

The dynamic priority scheme demonstrated its intended behaviour in both planar and Panda simulations. In the planar case, the cosine blending weight α rose through its full range as the elbow approached the ceiling, and the 9% EE error improvement over fixed priority confirmed that selective priority activation outperforms permanent priority assignment. In the Panda simulation, the lower null-space conflict ($\|J_2 P\| = 0.099$) reduced the blending weight to a maximum of 0.315, reflecting the geometric property that secondary tasks require less explicit priority management in higher-redundancy systems.

The dynamic priority scheme was not tested on the physical robot during the laboratory sessions due to time constraints. This represents the primary remaining validation gap and is identified as the most important direction for future work.

A key unresolved question is whether the 9% improvement observed in the planar simulation would replicate on the physical Panda. The simulation results suggest it would be smaller, given the larger null space reducing inter-task conflict. However, in configurations where all four redundant degrees of freedom are occupied by other constraints — for example, a scenario with three simultaneous Cartesian constraints plus joint limits — the priority blending mechanism would activate more aggressively and the improvement over fixed priority would be expected to recover. Testing this multi-constraint scenario on the physical Panda is identified as the primary direction for future experimental work.

7.5 Comparison with Existing Methods

The SNS algorithm comparison in planar Milestone 2 directly reproduced the failure modes described in Flacco et al. (2015): naive clamping produced 121 mm path deviation, task scaling produced a permanent 204 mm final error, and SNS alone achieved zero final error. These results match the qualitative descriptions in the paper and confirm that the implementation is faithful to the original algorithm.

The GSNS results are consistent with the claims of Kazemipour et al. (2022): joint and Cartesian constraints are treated identically, constraints can be enforced without parameter tuning, and the algorithm maintains a computational advantage over QP-based alternatives. The 18 μ s execution time reported in the paper for a KUKA LWR IV is not directly comparable to the Python implementation used here, but the algorithm structure is identical.

The dynamic priority contribution extends the SNS family in a direction not addressed by existing work. Mansard et al. (2009) and Simetti and Casalino (2016) both treat task priority as a discrete property — a task is either in the stack or not. The cosine blending mechanism introduced here continuously varies the relative priority weight based on robot state, eliminating velocity discontinuities at both priority entry and exit transitions.

The important point about the 9% improvement in EE tracking in the dynamic priority result is what is and is not claimed by that figure. It is not claimed that the dynamic priority policy

will always perform better than the fixed priority policy; if the elbow was to remain permanently in the activation region (for example, in the case of a robot moving perpetually near an edge or other geometric constraint), the fixed priority and the dynamic priority policies will essentially become the same because α will stay close to 1. What happens is that, during the trajectory of the arm, the elbow enters and leaves the activation region, which allows for a period in which α comes back to 0 and Task 1 gets the control over the null space. The 9% improvement figure is therefore specific to this geometry. What is guaranteed by the dynamic priority policy, no matter the task geometry, is the lack of velocity jumps at the transition between tasks, along with the activation function based on physics.

7.6 Publication Potential and Future Directions

The results of this project, particularly the novel dynamic task priority scheme and its experimental validation on the Franka Panda, represent a contribution of sufficient novelty and rigour for consideration at a peer-reviewed robotics venue. The combination of a verified simulation study across two robot platforms, a novel blending mechanism with theoretical justification, and physical experimental validation follows the structure expected of a conference paper at venues such as the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). Post-submission work will focus on three directions. First, resolving the coordinate frame offset between the Python kinematics implementation and the robot's internal model to enable precise absolute EE position targeting. Second, implementing the dynamic priority scheme on the physical Panda to complete the full experimental validation chain. Third, extending the cosine blending function to handle multiple simultaneous task priorities with overlapping activation zones, which is the natural generalisation required for complex human-robot interaction scenarios.

8. Conclusion

8.1 Summary of Contributions

The present dissertation provides an implementation, simulation and experimental demonstration of the Saturation in the Null Space (SNS) algorithm, its Generalised version (GSNS) and a new task priority approach for controlling kinematically redundant robots.

Four simulation results on a planar 3R manipulator in MATLAB have been provided, laying down the algorithmic basis. It has been shown that the SNS algorithm is the sole method among the four candidates that ensures the preservation of end-effector geometric properties of the path, as well as its completion under the constraints of joint velocity limits, resulting in zero final error and a 0.034 mm path deviation as opposed to 121 mm with naive clamping. The GSNS method has enforced a hard Cartesian ceiling constraint of $y = 1.55\text{ m}$ via the augmented matrix approach, whereas joint constraints allowed reaching $y = 1.79\text{ m}$. A priority scheme with dynamic adjustment has lowered EE tracking error by 9% in comparison with static priorities without reducing the level of constraint protection, which is provided by activation of Task 2 priority only when elbow approaching the dangerous zone.

All four algorithms have then been implemented on the 7-DOF Franka Emika Panda robot in simulation. The dimension agnostic structure of SNS and GSNS implementations has not

necessitated any structural changes for the Panda case, indicating the validity of the algorithm's generalisation to higher-dimensional spatial robots. A notable observation from the Panda results is that, in the scenarios tested, secondary constraints partially resolved in the null space ($\|J_2P\| = 0.099$), thus minimising conflicts between tasks and the necessity to implement task priorities.

Finally, physical experiments have been conducted on the Franka Research 3 robot with the application of the GSNS algorithm. The elbow y-coordinate has been kept below 0.30 m in four consecutive experimental trials, with the maximum observed value of 0.2274 m , 72.8 mm mean safety margin and 0.2 mm standard deviation across the trials. The GSNS algorithm does not require any parameter tuning between simulation and experiments due to the practical robustness of the augmented constraints formulation.

8.2 Limitations

There are several limitations to the presented work. These limitations do not coincide with the limitations of specific experiments described in Section 6.5 and are related to the scope and methodology of the dissertation.

First, the simulation platform used in all simulations is the planar 3R robot, and Franka Panda kinematic model. In other words, the kinematic properties of the simulation robot are rigid-body kinematics, without joint elasticity, backlash and motor dynamics. Actual manipulators possess elasticity of their joints at high speeds and high loads, which leads to actual velocities being different from commanded velocities. As the SNS algorithm works with the latter, the guarantee of constraints enforcement does not hold true in case of difference in velocities due to motor dynamics. Such an assumption is usual in the field of kinematic control (Flacco et al., 2015; Kazemipour et al., 2022).

The experimental validation included only the GSNS algorithm with a single Cartesian constraint. In particular, the dynamic priority scheme – the novel contribution of this project – was validated only in simulation. As a result, there is a mismatch between theoretical and experimental results: the 9% EE error improvement over fixed priority is a result of simulation which is yet to be verified independently on hardware. Multi-constraint situations described in Section 7.4 and motivating the priority blending algorithm have not been validated in any way.

The Python control system is working at 100 Hz , not the native 1000 Hz of the real-time interface of the Panda. In other words, the control loop is one order of magnitude slower compared to the actual robot dynamics, which significantly reduces the accuracy of trajectory tracking, as well as the possibility of safely testing fast movements that require fast constraint reactions. Due to the offset between the Python kinematics coordinate frame and the robot's coordinate frame (which is approximately 0.585 m), absolute EE target positions are currently inaccessible, limiting the experiment to relative EE motion commands only.

The used cosine blending function for dynamic priority scheme was designed and validated specifically for two simultaneous tasks and one scalar proximity signal. Extension to more than two tasks and more than one activation zone would require more general blending

architecture – the current implementation is a proof-of-concept, not a ready-to-use multi-task priority manager.

8.3 Recommendations for Further Work

Based on this research's results and limitations, five further research directions can be proposed in the order of importance.

The main research direction is the validation of the dynamic priority approach on the physical Franka Panda robot. The simulation shows that there is an improvement in EE error by 9% in comparison with fixed priority, but it should be proved in the real world on account of the fact that motor dynamics, noise from sensors, and the delay of the control loop should not have any influence on the result. One possible experiment can include a reproduction of Milestone 4 task in 3D workspace with a physical Python loop.

The second improvement relates to addressing the coordinate frame offset issue between the Python kinematics implementation and the robot's internal representation. The discrepancy originates in the difference in the definition of coordinate frames according to the standard DH convention that is used in this work, compared to the Franka's internal frame assignment, where there is a non-standard base frame rotation. Such calibration based on the factory forward kinematics will facilitate absolute EE position targeting and the common operation of the control pipeline — trajectory generation, SNS velocity computation, and constraint satisfaction — in the same coordinate frame.

Thirdly, increasing the frequency of the Python control loop to the native 1 kHz of the robot by interfacing directly with the C++ *libfranka* library would allow to achieve a higher level of precision and speed in the experiments. To get close to $18\ \mu s$ per step execution time reported by Kazemipour et al. (2022) and validate their real-time performance claim in Section 7.1, a 1 kHz control rate should be reached.

Fourthly, the extension of the dynamic prioritization algorithm to address multiple tasks with overlapping activation zones in parallel, is the natural generalization to make when applying the algorithm in human-robot collaboration scenarios. For instance, in case of three tasks, where the first is the main task of EE tracking and other two are constraints on the elbow and wrist orientation, respectively, the blending weight should be represented as a vector and each of its components should represent the proximity to the corresponding constraint boundary. Cosine profile can be easily extended to this case.

Finally, conducting the tests on the second robot platform, such as the KUKA LWR IV used in Kazemipour et al. (2022), would allow to verify independently the dimension-agnostic property shown in Section 5.7. A comparison of the execution time on the same hardware as in the referenced paper will allow to validate their $18\ \mu s$ claim as well.

8.4 Concluding Remarks

The project has shown that the SNS family of methods offers a computationally efficient, parameter-free, and hardware-transferable way of handling kinematic control for constrained redundant robots. The GSNS algorithm was the focus of the physical experimental validation and dynamic priority was only validated via simulation in two robots and is pending physical validation before it can be published. The dynamic priority

enhancement helps fill a specific gap in current literature and offers a solid base for adaptive human-robot interaction regarding task management in a safe manner. As can be seen from the results of the experimental evaluation using real hardware, even though rather limited, the algorithm shows hardware transferability without any modifications, which is the most important feature of a constraint handling method.

As far as the submission for publication outside of the dissertation is concerned, the results are novel and rigorous enough for a publication in a conference on robotics. The combination of a novel algorithm derived from a theoretical motivation, its machine-precision validation in simulation on two robot platforms, as well as experimental evaluation using real hardware follows the structure of the typical conference paper in venues like IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2027). After the submission is completed, post-submission work will include experimental validation of the dynamic priority scheme, resolution of the coordinate frame offset, and the development of a cosine blending scheme for multiple overlapping zones.

References

- Chan, T.F. and Dubey, R.V. (1995) A weighted least-norm solution-based scheme for avoiding joint limits for redundant joint manipulators. *IEEE Transactions on Robotics and Automation*, 11(2), pp. 286–292.
- Escande, A., Mansard, N. and Wieber, P.-B. (2014) Hierarchical quadratic programming: Fast online humanoid-robot motion generation. *International Journal of Robotics Research*, 33(7), pp. 1006–1028.
- Flacco, F., De Luca, A. and Khatib, O. (2012) Motion control of redundant robots under joint constraints: Saturation in the null space. In: *Proceedings of the IEEE International Conference on Robotics and Automation*, pp. 285–292.
- Flacco, F., De Luca, A. and Khatib, O. (2015) Control of redundant robots under hard joint constraints: Saturation in the null space. *IEEE Transactions on Robotics*, 31(3), pp. 637–654.
- Gaz, C., Cognetti, M., Oliva, A., Robuffo Giordano, P. and De Luca, A. (2019) Dynamic identification of the Franka Emika Panda robot with retrieval of feasible parameters using penalty-based optimization. *IEEE Robotics and Automation Letters*, 4(4), pp. 4147–4154.
- Kanoun, O., Lamiriaux, F. and Wieber, P.-B. (2011) Kinematic control of redundant manipulators: Generalizing the task-priority framework to inequality task. *IEEE Transactions on Robotics*, 27(4), pp. 785–792.
- Kazemipour, A., Khatib, M., Al Khudir, K., Gaz, C. and De Luca, A. (2022) Kinematic control of redundant robots with online handling of variable generalized hard constraints. *IEEE Robotics and Automation Letters*, 7(4), pp. 9279–9286.
- Khatib, O. (1986) Real-time obstacle avoidance for manipulators and mobile robots. In: Cox, I. and Wilfong, G. (eds.) *Autonomous Robot Vehicles*. Berlin: Springer, pp. 396–404.
- Khatib, M., Al Khudir, K. and De Luca, A. (2020) Task priority matrix at the acceleration level: Collision avoidance under relaxed constraints. *IEEE Robotics and Automation Letters*, 5(3), pp. 4970–4977.
- Liegeois, A. (1977) Automatic supervisory control of the configuration and behavior of multibody mechanisms. *IEEE Transactions on Systems, Man, and Cybernetics*, 7(12), pp. 868–871.
- Mansard, N., Khatib, O. and Kheddar, A. (2009) A unified approach to integrate unilateral constraints in the stack of tasks. *IEEE Transactions on Robotics*, 25(3), pp. 670–685.
- Siciliano, B. and Slotine, J.J. (1991) A general framework for managing multiple tasks in highly redundant robotic systems. In: *Proceedings of the 5th International Conference on Advanced Robotics*, pp. 1211–1216.
- Simetti, E. and Casalino, G. (2016) A novel practical technique to integrate inequality control objectives and task transitions in priority-based control. *Journal of Intelligent and Robotic Systems*, 84(1), pp. 877–902.

Appendices

Appendix A — MATLAB File Summary

File	Milestone	Description
planar_3R_forward_kinematics.m	1	Forward kinematics, Jacobian derivation, pseudoinverse properties, null-space verification
planar_3R_line_tracking.m	1	Straight-line tracking simulation, three null-space strategies compared
planar_3R_animation.m	1	Animated visualisation of null-space reconfiguration
planar_3R_saturation_problem_v2.m	2	Demonstration of saturation problem: no limits, naive clamping, task scaling
planar_3R_SNS.m	2	Full SNS algorithm (Flacco Algorithm 1 and 2), four-method comparison
planar_3R_GSNS.m	3	GSNS algorithm (Kazemipour Algorithm 1), joint and Cartesian constraint enforcement
planar_3R_dynamic_priorities.m	4	Dynamic task priority scheme, cosine blending function, three-method comparison

All files use $T = 0.001$ s sampling, quintic polynomial timing, and the feedback control law from Kazemipour et al. (2022) Equation 15. Full source code is available in the shared OneDrive folder under MSc_Project/code/.

Franka Panda files (Chapters 5 and 6)

File	Milestone	Description
panda_fk.m	Panda M1	Forward kinematics, standard DH, 7-DOF Jacobian, verification
panda_jacobian.m	Panda M1	Geometric Jacobian 6×7 , verified against finite differences
panda_elbow_jacobian_y.m	Panda M3/M4	1×7 Jacobian of elbow y-coordinate

verify_panda_kinematics.m	Panda M1	Five-test verification suite
SNS_algorithm.m	Panda M2	SNS algorithm (unchanged from planar, dimension-agnostic)
GSNS_algorithm.m	Panda M3	GSNS algorithm (unchanged from planar, dimension-agnostic)
panda_SNS_test.m	Panda M2	3D straight-line tracking with SNS on Panda
panda_GSNS_test.m	Panda M3	GSNS elbow ceiling constraint on Panda
panda_dynamic_priorities_test.m	Panda M4	Dynamic priority scheme on Panda
panda_fk_python.py	Ch 6	Python forward kinematics for physical robot
panda_sns_python.py	Ch 6	Python SNS algorithm for physical robot
panda_robot_control.py	Ch 6	Integrated Python control loop for Franka Research 3
dyn_priority_stack.m		Dynamic priority blending function – computes q_{12} and q_{21} and blends with weight α

Appendix B — Quantitative Results Summary

Milestone 2 — SNS comparison (joint velocity limits: [0.3, 0.4, 0.5] rad/s)

Method	Max error (mm)	Path deviation (mm)	Final error (mm)	Saturation events
No limits (ideal)	0.01	0.002	0.000	0
Naive clamping	344.3	121.3	0.012	3336
Task scaling	336.8	0.033	203.840	4739
SNS algorithm	212.2	0.034	0.000	1473

Milestone 3 — GSNS Cartesian constraint (ceiling at $y = 1.55$ m)

Method	Max elbow y (m)	Constraint satisfied

No limits	1.7935	No
SNS (joint only)	1.7935	No
GSNS (joint + Cartesian)	1.5500	Yes

Milestone 4 — Dynamic task priorities (ceiling at $y = 1.60$ m)

Method	Max EE error (mm)	Max elbow y (m)	Ceiling respected
Task 1 only	158.1	1.6438	No
Fixed priority	256.3	1.4221	Yes
Dynamic priority	233.2	1.4212	Yes

Appendix C — Key Equations Reference

The following equations from the primary references are implemented directly in the MATLAB code and cited throughout this report:

- Flacco et al. (2015) Eq. 1: differential kinematics $\dot{\mathbf{x}} = \mathbf{J}\dot{\mathbf{q}}$
- Flacco et al. (2015) Eq. 2: general redundancy resolution $\dot{\mathbf{q}} = \mathbf{J}^\# \dot{\mathbf{x}} + \mathbf{P}\dot{\mathbf{q}}_N$
- Flacco et al. (2015) Eqs. 12–15: joint velocity constraint shaping
- Flacco et al. (2015) Eq. 16: SNS projection equation
- Kazemipour et al. (2022) Eqs. 7–8: augmented constraint vector and matrix
- Kazemipour et al. (2022) Eq. 6: Cartesian constraint shaping
- Kazemipour et al. (2022) Eq. 15: feedback control law with proportional gain
- Gaz et al. (2019) Fig. 1: Franka Panda DH parameters — used for Panda kinematic model in Chapters 5 and 6

Appendix D — Raw Experimental Data from Physical Panda

Table D.1: Joint angles recorded during Experiment 1 (arbitrary configuration)

$\mathbf{q} = [1.39 \times 10^{-3}, 1.47 \times 10^{-1}, 3.38 \times 10^{-1}, -2.44, -1.89 \times 10^{-1}, 2.66, 1.02]$ rad

Table D.2: Joint angles achieved at ready pose

$\mathbf{q} = [2.80 \times 10^{-4}, -7.84 \times 10^{-1}, 2.12 \times 10^{-4}, -2.36, 7.05 \times 10^{-5}, 1.57, 7.85 \times 10^{-1}]$ rad

Table D.3: GSNS constraint experiment — elbow y-coordinate across four runs

Run	Initial (m)	Maximum (m)	Ceiling (m)	Margin (m)
1	0.2265	0.2270	0.30	0.0730
2	0.2266	0.2271	0.30	0.0729
3	0.2266	0.2273	0.30	0.0727
4	0.2274	0.2274	0.30	0.0726

Mean maximum elbow y: 0.2272 m. Standard deviation: 0.0002 m. All runs satisfied the ceiling constraint with a mean safety margin of 72.8 mm.

Appendix E — MATLAB Algorithm Source Code

The following MATLAB functions implement the core algorithms described in Chapters 3 and 4. Test and simulation scripts are not included; the files below contain the algorithm logic only. All files are available in the shared OneDrive folder under MSc_Project/franka_panda/ and MSc_Project/planar_3R/.

Geometric Jacobian for Franka Panda (6x7)

```

1  function J = panda_jacobian(q)
2  % Geometric Jacobian for Franka Panda (6x7)
3  % Verified against finite differences: error < 2e-8
4  %
5  % Input:  q - 7x1 joint angles in radians
6  % Output: J - 6x7 Jacobian matrix
7
8  [T_ee, T_joints] = panda_fk(q);
9
10 p_ee = T_ee(1:3, 4);
11 J     = zeros(6, 7);
12
13 for i = 1:7
14     z = T_joints{i}(1:3, 3);
15     p = T_joints{i}(1:3, 4);
16     J(1:3, i) = cross(z, p_ee - p);
17     J(4:6, i) = z;
18 end
19 end

```

```

1  function [q_dot, s_star, iterations] = SNS_algorithm(J, x_dot, Vmin, Vmax, n, m)
2
3  W = eye(n);
4  q_dot_N = zeros(n,1);
5  s = 1;
6  s_star = 0;
7  q_dot_star = zeros(n,1);
8  iterations = 0;
9  max_iter = n + 1;
10 limit_exceeded = true;
11
12 while limit_exceeded && iterations <= max_iter
13     iterations = iterations + 1;
14     limit_exceeded = false;
15     JW = J * W;
16     if rank(JW) < m
17         break;
18     end
19     JW_pinv = pinv(JW);
20     q_dot = q_dot_N + JW_pinv * (s * x_dot - J * q_dot_N);
21     if all(q_dot >= Vmin - 1e-10) && all(q_dot <= Vmax + 1e-10)
22         s_star = s;
23         q_dot_star = q_dot;
24         break;
25     end
26     a = JW_pinv * x_dot;
27     b = q_dot - s * a;
28     s_values = ones(n,1);
29     for i = 1:n
30         if abs(a(i)) < 1e-10
31             if b(i) < Vmin(i)-1e-10 || b(i) > Vmax(i)+1e-10
32                 s_values(i) = 0;
33             end
34         else
35             s_lo = (Vmin(i)-b(i))/a(i);
36             s_hi = (Vmax(i)-b(i))/a(i);
37             if s_lo > s_hi; tmp=s_lo; s_lo=s_hi; s_hi=tmp; end
38             if s_hi < 0 || s_lo > 1
39                 s_values(i) = 0;
40             else
41                 s_values(i) = min(1, max(0, s_hi));
42             end
43         end
44     end
45     [s_k, j] = min(s_values);
46     if s_k > s_star
47         s_star = s_k;
48         q_dot_star = b + s_k * a;
49     end
50     W_test = W; W_test(j,j) = 0;
51     if rank(J * W_test) < m
52         break;
53     end
54     W(j,j) = 0;
55     if q_dot(j) > Vmax(j)
56         q_dot_N(j) = Vmax(j);
57     else
58         q_dot_N(j) = Vmin(j);
59     end
60     limit_exceeded = true;
61 end
62
63 q_dot = max(Vmin, min(Vmax, q_dot_star));
64 end

```

```

1 function [q_dot, s_star] = GSNS_algorithm(J, x_dot, ...
2     Vmin_j, Vmax_j, J_cp, cp_bmin, cp_bmax, cp_pos, q, L, n, m)
3
4 has_cartesian = ~isempty(J_cp);
5
6 if has_cartesian
7     d_cp = size(J_cp, 1);
8     n_constraints = n + d_cp;
9     A = [eye(n); J_cp];
10    bmin = [Vmin_j; cp_bmin];
11    bmax = [Vmax_j; cp_bmax];
12 else
13     n_constraints = n;
14     A = eye(n);
15     bmin = Vmin_j;
16     bmax = Vmax_j;
17     d_cp = 0;
18 end
19
20 q_dot_N = zeros(n,1);
21 s_star = 0;
22 P = eye(n);
23 Alim = [];
24 a_dot_N = [];
25 q_dot_star = zeros(n,1);
26 max_iter = n + d_cp;
27
28 for iter = 1:max_iter
29     JP = J * P;
30     if rank(JP, 1e-6) < m; break; end
31     JP_pinv = pinv(JP);
32     q_dot = q_dot_N + JP_pinv * (x_dot - J * q_dot_N);
33     a_dot = A * q_dot;
34     violations = (a_dot < bmin - 1e-10) | (a_dot > bmax + 1e-10);
35     if ~any(violations)
36         s_star = 1;
37         q_dot_star = q_dot;
38         break;
39     end
40     alpha = A * JP_pinv * x_dot;
41     beta = a_dot - alpha;
42     [s_k, k_crit] = getTaskScalingFactor_GSNS(alpha, beta, bmin, bmax, n_constraints);
43     if s_k > s_star
44         s_star = s_k;
45         q_dot_star = max(Vmin_j, min(Vmax_j, ...
46             q_dot_N + JP_pinv * (s_k * x_dot - J * q_dot_N)));
47     end
48     Ak = A(k_crit,:);
49     if a_dot(k_crit) > bmax(k_crit)
50         sat_val = bmax(k_crit);
51     else
52         sat_val = bmin(k_crit);
53     end
54     Alim = [Alim; Ak];
55     a_dot_N = [a_dot_N; sat_val];
56     P = eye(n) - pinv(Alim) * Alim;
57     if rank(J*P, 1e-6) < m
58         if size(Alim,1) > 1
59             Alim = Alim(1:end-1,:);
60             a_dot_N = a_dot_N(1:end-1);
61             P = eye(n) - pinv(Alim) * Alim;
62         end
63         break;
64     end
65     q_dot_N = pinv(Alim) * a_dot_N;

```

```

66 end
67
68 q_dot = max(Vmin_j, min(Vmax_j, q_dot_star));
69 end
70
71 function [s, k_crit] = getTaskScalingFactor_GSNS(alpha, beta, bmin, bmax, nc)
72 s_values = ones(nc,1);
73 for h = 1:nc
74     L_h = bmin(h) - beta(h);
75     U_h = bmax(h) - beta(h);
76     if alpha(h) < 0 && L_h < 0
77         if alpha(h) < L_h
78             s_values(h) = L_h / alpha(h);
79         else
80             s_values(h) = 1;
81         end
82     elseif alpha(h) > 0 && U_h > 0
83         if alpha(h) > U_h
84             s_values(h) = U_h / alpha(h);
85         else
86             s_values(h) = 1;
87         end
88     else
89         s_values(h) = 0;
90     end
91 end
92 [s, k_crit] = min(s_values);
93 end

```

```

1 function [T_ee, T_joints] = panda_fk(q)
2 % Franka Panda forward kinematics - Standard DH convention
3 % Verified: Jacobian error < 2e-8, ||JP|| < 2e-14
4 %
5 % Standard DH: T = Rot_z(theta)*Trans_z(d)*Trans_x(a)*Rot_x(alpha)
6 % Parameters: [a, d, alpha]
7
8 DH = [0,      0.333,  0;
9       0,      0,      -pi/2;
10      0,      0.316,  pi/2;
11      0.0825,  0,      pi/2;
12      -0.0825, 0.384, -pi/2;
13      0,      0,      pi/2;
14      0.088,  0.107,  pi/2];
15
16 T_joints = cell(1,8);
17 T_joints{1} = eye(4);
18 T_current = eye(4);
19
20 for i = 1:7
21     a = DH(i,1);
22     d = DH(i,2);
23     alpha = DH(i,3);
24     th = q(i);
25
26     Ti = [cos(th), -sin(th)*cos(alpha), sin(th)*sin(alpha), a*cos(th);
27           sin(th), cos(th)*cos(alpha), -cos(th)*sin(alpha), a*sin(th);
28           0,      sin(alpha),      cos(alpha),      d;
29           0,      0,      0,      1];
30
31     T_current = T_current * Ti;
32     T_joints{i+1} = T_current;
33 end
34
35 T_ee = T_current;
36 end

```

```

1 function q_dot = dyn_priority_stack(J1,xd1,J2,xd2,Vmin,Vmax,n,m1,alpha)
2
3 % When alpha=0: pure Task 1 only (no Task 2 influence at all)
4 if alpha < 1e-6
5     [q_dot,~] = SNS_velocity_p(J1,xd1,Vmin,Vmax,n,m1);
6     q_dot = max(Vmin, min(Vmax, q_dot));
7     return;
8 end
9
10 % When alpha=1: pure Task 2 priority
11 if alpha > 1-1e-6
12     [qd_t2,~] = SNS_velocity_p(J2,xd2,Vmin,Vmax,n,1);
13     P2 = eye(n) - pinv(J2)*J2;
14     J1p = J1*P2;
15     if rank(J1p,1e-6) >= m1
16         q_dot = qd_t2 + pinv(J1p)*(xd1 - J1*qd_t2);
17     else
18         q_dot = qd_t2;
19     end
20     q_dot = max(Vmin, min(Vmax, q_dot));
21     return;
22 end
23
24 % Intermediate alpha: compute both orderings and blend
25 % Ordering 1: Task 1 > Task 2
26 [qd_t1,~] = SNS_velocity_p(J1,xd1,Vmin,Vmax,n,m1);
27 P1 = eye(n) - pinv(J1)*J1;
28 J2p = J2*P1;
29 if rank(J2p,1e-6) >= 1
30     qd_12 = qd_t1 + pinv(J2p)*(xd2 - J2*qd_t1);
31 else
32     qd_12 = qd_t1;
33 end
34 qd_12 = max(Vmin, min(Vmax, qd_12));
35
36 % Ordering 2: Task 2 > Task 1
37 [qd_t2,~] = SNS_velocity_p(J2,xd2,Vmin,Vmax,n,1);
38 P2 = eye(n) - pinv(J2)*J2;
39 J1p = J1*P2;
40 if rank(J1p,1e-6) >= m1
41     qd_21 = qd_t2 + pinv(J1p)*(xd1 - J1*qd_t2);
42 else
43     qd_21 = qd_t2;
44 end
45 qd_21 = max(Vmin, min(Vmax, qd_21));
46
47 % Blend
48 q_dot = (1-alpha)*qd_12 + alpha*qd_21;
49 q_dot = max(Vmin, min(Vmax, q_dot));
50 end

```

Appendix F — Python Experimental Implementation

The following Python files implement the control pipeline deployed on the physical Franka Research 3 during the experimental session described in Chapter 6. The `pyfranka_interface` library, pre-installed on the laboratory laptop, provides the robot communication layer.

```
panda_robot_control.py

1  #!/usr/bin/env python3
2  """
3  Panda SNS Control Script
4  Smart Oluwatobi - K2457624
5  MSc Mechatronics - Kingston University
6
7  Connects to Franka Panda and runs SNS velocity control
8  """
9
10 import pyfranka_interface as franka
11 import numpy as np
12 import time
13
14 # Import your algorithms
15 from panda_fk_python import panda_fk, panda_jacobian, get_elbow_y
16 from panda_sns_python import sns_algorithm
17
18 # ===== PARAMETERS =====
19 ROBOT_IP = "192.168.1.13"
20 Vmax_val = 0.1 # conservative for first test - 10% of max
21 dt = 0.01 # 10ms steps (100Hz - safer than 1kHz for first test)
22 Kp = 20 # proportional gain
23
24
25 def test_read_joints():
26     """Step 1: Just read and print joint angles"""
27     print("Connecting to robot...")
28     r = franka.Robot(ROBOT_IP, False, False)
29     state = r.getState()
30     q = np.array(state.q)
31     print("Current joint angles:", q)
32     print("EE transform:")
33     print(state.T)
34
35     # Verify with your kinematics
36     T_ee, _ = panda_fk(q)
37     print("Your FK result:", T_ee[:3, 3])
38     print("Robot FK result:", np.array(state.T)[:3, 3])
39     print("Difference:", np.linalg.norm(T_ee[:3, 3] - np.array(state.T)[:3, 3]))
40     return r, q
41
42
43 def move_to_joint_position(r, q_target, speed=0.15):
44     """Step 2: Move to a joint configuration"""
45     print(f"Moving to target joints at speed {speed}...")
46     r.move_joints(np.array(q_target), speed)
47     state = r.getState()
48     print("Reached:", np.array(state.q))
49
50
51 def sns_straight_line(r, delta_position, duration=5.0):
52     """
53     Step 3: SNS velocity control for straight-line EE motion
54     delta_position: [dx, dy, dz] in metres - how far to move
55     """
56     Vmin = -Vmax_val * np.ones(7)
57     Vmax = Vmax_val * np.ones(7)
58
59     # Get start state
60     state = r.getState()
61     q = np.array(state.q)
62     T_ee, _ = panda_fk(q)
63     p_start = T_ee[:3, 3]
64     p_target = p_start + np.array(delta_position)
65
66     print(f"SNS straight line: {p_start} -> {p_target}")
67     print(f"Duration: {duration}s")
```

```

69 start_time = time.time()
70 step = 0
71
72 while time.time() - start_time < duration:
73     # Read current state
74     state = r.getState()
75     q = np.array(state.q)
76
77     # Current EE position from robot FK
78     T_robot = np.array(state.T).reshape(4, 4)
79     p_curr = T_robot[:3, 3]
80
81     # Desired position - quintic timing
82     t = time.time() - start_time
83     tau = min(t / duration, 1.0)
84     gam = 6 * tau**5 - 15 * tau**4 + 10 * tau**3
85     gam_dot = (30 * tau**4 - 60 * tau**3 + 30 * tau**2) / duration
86     p_des = p_start + gam * np.array(delta_position)
87     v_des = gam_dot * np.array(delta_position)
88
89     # Task velocity with feedback
90     x_dot = v_des + Kp * (p_des - p_curr)
91
92     # Limit speed
93     speed = np.linalg.norm(x_dot)
94     if speed > 0.03: # max 3cm/s
95         x_dot = x_dot * 0.03 / speed
96
97     # Jacobian - position rows only
98     J = panda_jacobian(q)
99     J_pos = J[:3, :]
100
101     # SNS algorithm
102     q_dot, s = sns_algorithm(J_pos, x_dot, Vmin, Vmax)
103
104     # Send velocity command
105     # NOTE: move_joints takes position not velocity
106     # We integrate to get next joint position
107     q_next = q + q_dot * dt
108     r.move_joints(q_next, 0.1)
109
110     # Print progress every 50 steps
111     if step % 50 == 0:
112         err = np.linalg.norm(p_des - p_curr) * 1000
113         elbow_y = get_elbow_y(q)
114         print(f"t={t:.1f}s err={err:.1f}mm s={s:.3f} elbow_y={elbow_y:.4f}m")
115
116     step += 1
117     time.sleep(dt)
118
119 # Final state
120 state = r.getState()
121 q_final = np.array(state.q)
122 T_final = np.array(state.T).reshape(4, 4)
123 print(f"Final EE position: {T_final[:3, 3]}")
124 print(f"Final error: {np.linalg.norm(T_final[:3, 3] - p_target) * 1000:.2f} mm")
125
126
127 def gsns_constrained_motion(r, delta_position, y_ceiling=0.30, duration=5.0):
128     """
129     Step 4: GSNS with elbow y ceiling constraint
130     """
131     from panda_fk_python import panda_elbow_jacobian_y
132
133     Vmin = -Vmax_val * np.ones(7)
134     Vmax_arr = Vmax_val * np.ones(7)
135
136     # GSNS augmented system
137     def build_augmented(J_pos, q, y_ceiling, y_floor=-0.5):
138         J_elbow = panda_elbow_jacobian_y(q)
139         elbow_y = get_elbow_y(q)

```

```

141     # Shape Cartesian constraint
142     cp_Vmax = 0.1
143     cp_Amax = 0.5
144     v_pos_max = (y_ceiling - elbow_y) / dt
145     v_pos_min = (y_floor - elbow_y) / dt
146     margin = y_ceiling - elbow_y
147     v_stop = np.sqrt(2 * cp_Amax * margin) if margin > 0 else 0
148     bmax = min(v_pos_max, cp_Vmax, v_stop)
149     bmin = max(v_pos_min, -cp_Vmax)
150     if bmax < 0:
151         bmax = 0
152     if bmin > 0:
153         bmin = 0
154
155     # Augmented matrix
156     A = np.vstack([np.eye(7), J_elbow])
157     b_min = np.append(Vmin, bmin)
158     b_max = np.append(Vmax_arr, bmax)
159     return A, b_min, b_max, elbow_y
160
161 state = r.getState()
162 q = np.array(state.q)
163 T_start = np.array(state.T).reshape(4, 4)
164 p_start = T_start[:3, 3]
165 p_target = p_start + np.array(delta_position)
166
167 print(f"GSNS motion with elbow ceiling at y={y_ceiling}m")
168 print(f"Initial elbow y: {get_elbow_y(q):.4f}m")
169
170 start_time = time.time()
171 max_elbow_y = get_elbow_y(q)
172
173 while time.time() - start_time < duration:
174     state = r.getState()
175     q = np.array(state.q)
176     T_robot = np.array(state.T).reshape(4, 4)
177     p_curr = T_robot[:3, 3]
178
179     t = time.time() - start_time
180     tau = min(t / duration, 1.0)
181     gam = 6 * tau**5 - 15 * tau**4 + 10 * tau**3
182     gam_dot = (30 * tau**4 - 60 * tau**3 + 30 * tau**2) / duration
183     p_des = p_start + gam * np.array(delta_position)
184     v_des = gam_dot * np.array(delta_position)
185     x_dot = v_des + Kp * (p_des - p_curr)
186
187     speed = np.linalg.norm(x_dot)
188     if speed > 0.03:
189         x_dot = x_dot * 0.03 / speed
190
191     J = panda_jacobian(q)
192     J_pos = J[:3, :]
193
194     # GSNS with augmented constraints
195     A, b_min, b_max, elbow_y = build_augmented(J_pos, q, y_ceiling)
196     max_elbow_y = max(max_elbow_y, elbow_y)
197
198     # Use SNS on augmented system
199     q_dot, s = sns_algorithm(A[:3, :], x_dot, b_min[:7], b_max[:7])
200
201     q_next = q + q_dot * dt
202     r.move_joints(q_next, 0.1)
203
204     time.sleep(dt)
205
206 print(f"Max elbow y reached: {max_elbow_y:.4f}m")
207 print(f"Ceiling was: {y_ceiling}m")
208 if max_elbow_y > y_ceiling + 0.005:
209     print("CONSTRAINT VIOLATED")
210 else:
211     print("Constraint satisfied")

```

```

212
213
214 # ===== MAIN - RUN IN ORDER =====
215 if __name__ == "__main__":
216     print("=" * 50)
217     print("STEP 1: Read joint angles")
218     print("=" * 50)
219     r, q_init = test_read_joints()
220
221     input("\nPress Enter to move to ready pose...")
222     ready_pose = [0, -0.785, 0, -2.356, 0, 1.571, 0.785]
223     move_to_joint_position(r, ready_pose, speed=0.15)
224
225     input("\nPress Enter to run SNS straight line (2cm in x)...")
226     sns_straight_line(r, delta_position=[0.02, 0.0, 0.0], duration=3.0)
227
228     input("\nPress Enter to run GSNS constrained motion...")
229     gsns_constrained_motion(
230         r, delta_position=[0.0, 0.05, 0.05], y_ceiling=0.30, duration=5.0
231     )
232
233     print("\nSession complete.")

```